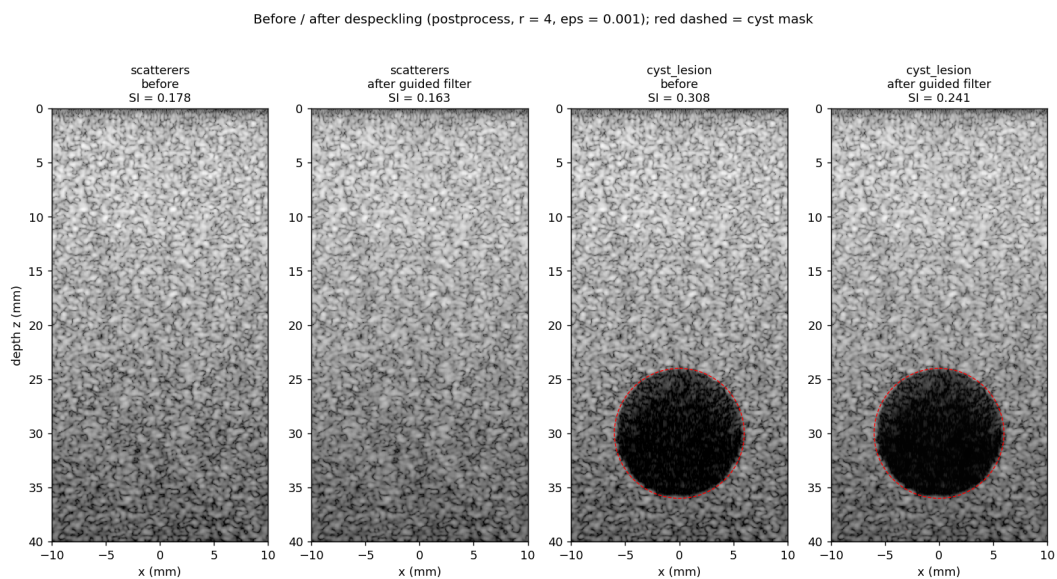


Team 8: Post-Image Processing

Cleaning the speckle from the finished grayscale B-mode image

NITK-UsoundSim · 2-D ultrasound simulator · explanation, input, output and run guide

- **Post-Image Processing:** a finished grayscale B-mode image -> guided filter (despeckling) -> quality metrics.
- Stage order in the pipeline: Image Reconstruction (Team 6) -> B-mode Image Formation (Team 7) -> **Post-Image Processing (this stage, last)**.
- One command, `python3 run_team8.py`, runs in a few seconds and saves all results in output/.
- Measured on the simulated cyst image: speckle index 0.308 -> 0.241, gCNR 0.860 -> 0.871, 96 % of the edge sharpness kept.



Result from `run_team8.py`: the two simulator images before and after the guided filter.

Contents

1 Overview	3
Files in the folder	3
2 Theory	4
2.1 what speckle is	4
2.2 why despeckle, and why a plain blur is bad	4
2.3 the guided filter (He et al. 2013)	4
2.4 the quality metrics	4
3 Code explanation	6
3.1 post_processing.py	6
3.2 262SP009_Post_Image_Processing_v2.ipynb (study notebook)	6
3.3 How run_team8.py uses the team's own code	7
3.4 run_team8.py (runner written for this demo)	8
3.5 Full listings	8
4 Input	27
5 Output	28
5.1 Quality metrics and figures	28
5.2 Console output	31
5.3 Results from the full study notebook (earlier run on real ultrasound datasets)	33
6 How to run	36
Opening the study notebook	36
Troubleshooting	36
7 Check yourself	37

1 Overview

What this stage does and which files are in the folder

The NITK-UsoundSim project simulates a 2-D ultrasound scanner as a chain of stages: transducer, transmit beamforming, wave propagation, tissue, receive beamforming, Image Reconstruction (Team 6), B-mode Image Formation (Team 7) and finally **Post-Image Processing**. This last stage starts only when the grayscale B-mode image exists: it reduces the speckle in that image while keeping edges, and measures the image quality.

The study notebook 262SP009_Post_Image_Processing_v2.ipynb compared 10 despeckling filters on real ultrasound images and picked the **guided filter**; the module `post_processing.py` applies it (`postprocess()`, $r = 4$, $\text{eps} = 0.001$) and gives the quality metrics.

Files in the folder

Path	What it is	Origin
<code>run_team8.py</code>	Runs the filter and the metrics and saves everything in output/	added for this demo
<code>code/post_processing.py</code>	The module: <code>postprocess()</code> , <code>guided()</code> , <code>to_uint8()</code> , <code>metrics</code> , and <code>scan_convert()</code>	team original, unchanged
<code>code/262SP009_Post_Image_Processing_v2.ipynb</code>	Full despeckling study on real datasets	team original, unchanged
<code>code/notebook_results/*.png</code>	10 figures saved by the study notebook	team original (earlier run)
<code>input/cyst_lesion_image.npz</code> , <code>input/scatterers_image.npz</code>	Sample grayscale B-mode images (512 x 256 px)	taken from the project's saved simulator results
<code>input/*_preview.png</code>	Pictures of the two inputs	added for this demo
<code>output/</code>	All results of <code>run_team8.py</code>	made by <code>run_team8.py</code>
<code>README.md</code> , <code>requirements.txt</code> , this PDF	Instructions	added for this demo

Note

`post_processing.py` also contains `scan_convert()` for the linear-array simulator; that function belongs to the B-mode image formation stage and is not part of this demo.

`code/post_processing.py` is a **library module**: it only defines functions and has no `if __name__ == "__main__":` block, so running it on its own does nothing useful. It does `import config` (the full project's settings file), which only `scan_convert()` uses. Always start the demo with `run_team8.py`.

2 Theory

The ideas behind the code, in simple words

The input of this stage is already a finished grayscale B-mode image. Image Reconstruction (Team 6) detected the envelope with the Hilbert transform and log-compressed it; B-mode Image Formation (Team 7) applied the 50 dB dynamic range and mapped it to grey levels 0-1. Post-image processing works only on these pixels.

2.1 what speckle is

Tissue contains very many tiny scatterers inside one resolution cell. Their echoes add with random phases: sometimes they add up (bright), sometimes they cancel (dark). The result is the grainy **speckle** pattern. It is not noise from the electronics; it is interference. When the envelope is the size of a sum of many random phasors, it follows a **Rayleigh distribution**:

$$p(E) = \frac{E}{\sigma^2} e^{-E^2/(2\sigma^2)}, \quad \frac{\text{std}(E)}{\text{mean}(E)} \approx 0.52$$

fully developed speckle: the spread is about half the mean, whatever the tissue brightness

2.2 why despeckle, and why a plain blur is bad

Speckle hides small, low-contrast lesions and makes borders ragged. Averaging nearby pixels reduces it, but a plain blur (box or Gaussian filter) averages *across* edges too, so the border of a cyst becomes soft and small structures disappear. We want a filter that smooths flat regions strongly and edges weakly.

2.3 the guided filter (He et al. 2013)

The guided filter assumes that in each small window w_k (size $(2r+1) \times (2r+1)$) the output is a straight-line function of the guide image I : $q = a_k I + b_k$. Here the guide is the image itself (self-guided). Choosing a_k, b_k by least squares gives:

$$a_k = \frac{\text{cov}_k(I, I)}{\text{var}_k(I) + \varepsilon} = \frac{\sigma_k^2}{\sigma_k^2 + \varepsilon}, \quad b_k = \bar{I}_k - a_k \bar{I}_k$$

$$q = \bar{a} I + \bar{b}$$

bars = average over all windows that cover the pixel (box filter)

- **Flat region** (local variance much smaller than ε): a is near 0, so q is near the local mean -> strong smoothing.
- **Edge** (local variance much larger than ε): a is near 1 and b near 0, so q is near I -> the edge is kept.
- **ε** sets the border between 'speckle' and 'edge'. Bigger ε = more smoothing, softer edges. **r** sets the window size ($r = 4$ -> 9×9 pixels, about 0.7 mm here).
- It uses only box filters, so it is very fast (a few milliseconds per image).

2.4 the quality metrics

There is no clean 'true' image, so the metrics compare regions of the same image. Grey levels are 0-255 (uint8).

$$SI = \text{mean}\left(\frac{\sigma_{7 \times 7}}{\mu_{7 \times 7}}\right)$$

speckle index: local spread / local mean; lower = smoother

$$CNR = \frac{|\mu_{lesion} - \mu_{ring}|}{\sqrt{\sigma_{lesion}^2 + \sigma_{ring}^2}}$$

contrast-to-noise ratio: lesion versus a 10-pixel ring around it; higher = easier to see

$$gCNR = 1 - \sum_g \min(h_{lesion}(g), h_{ring}(g))$$

generalised CNR: 1 minus the overlap of the two grey-level histograms; 0 = cannot be told apart, 1 = fully separable

$$\text{Edge} = \text{mean } |\nabla I|_{\text{Sobel}} \text{ on a 3 px band at the border}$$

edge sharpness; 'edge kept' = Edge after / Edge before

The runner also prints the **contrast in dB**: the images are log-compressed with a 50 dB dynamic range, so a difference of 1.0 in [0, 1] brightness equals 50 dB, and contrast = 50 x (mean lesion - mean ring).

3 Code explanation

Every file, section by section, then the full listings

3.1 post_processing.py

Section	What it does
Docstring	Describes inputs/outputs, the guided filter formula, why $r = 4$ and $\text{eps} = 0.001$ were chosen (study notebook), and limits.
import config	Project settings, needed only by scan_convert(). The runner supplies an empty placeholder if it is missing.
scan_convert()	Linear-array resampling for the B-mode team; not used in this demo.
_finish()	Round, clip to 0-255, convert to uint8; pixels outside the scan mask keep their input value.
guided()	The guided filter, copied bit-identically from the study notebook: box means , cov , var , $a = \text{cov}/(\text{var}+\text{eps})$, $b = \text{mean} - a \text{ mean}$, $\text{output} = \text{box}(a) G + \text{box}(b)$.
to_uint8()	[0, 1] float -> 0-255 grey levels (the notebook's working format).
postprocess()	Checks the input, converts to uint8, runs guided() on the whole image, returns float [0, 1]; optional gamma mapping (default 1 = none); method='none' skips the filter.
speckle_index()	Mean of local std / local mean (7 x 7) where the local mean > 1.
lesion_ring()	Lesion dilated by 10 px, minus the lesion: the background ring.
cnr(), gcnr()	CNR and gCNR between lesion and ring (formulas in Chapter 2).
edge_sharpness()	Mean Sobel gradient on a 3-px band across the lesion border.

3.2 262SP009_Post_Image_Processing_v2.ipynb (study notebook)

This notebook is the research behind this stage. It is written for Google Colab and needs internet. It is not run by run_team8.py; its saved figures are shown in Chapter 5.

Notebook section	What it does
Setup (cells 1-2)	pip installs kagglehub and bm3d; imports; seed 0; 20 images per organ, longest side 512 px.
1. Data loading (cells 4-9)	Downloads breast (BUSI), thyroid (TN3K), liver (Zenodo) and kidney (fallback Kaggle set, no masks); loads images and lesion masks; shows samples.
2. Preprocessing (cells 11-13)	auto_mask (scan area), find_annotations / clean_annotations (remove burned-in text and calipers by inpainting), normalize (percentile stretch).
3. Quality metrics (cells 15-17)	ENL, SI, CNR, gCNR, edge sharpness, runtime; automatic background ROI; baseline table.
4. Despeckle filters (cells 19-20)	10 filters with one signature: Median, Lee, SRAD, NLM, Wavelet, OBNLM, BM3D, Guided, Bilateral, TV; demo on one breast image.
5. Comparison on breast (cells 22-25)	10 tune / 10 test images; for each filter pick the parameter with best CNR while keeping edge ratio ≥ 0.90 ; rank by gCNR. Guided (eps 0.001) wins.
6. Improvements (cells 27-30)	Hybrids (wavelet + guided), CLAHE + sharpening, automatic eps from the noise level. None beats the plain guided filter clearly.
7. Final pipeline (cells 32-34)	Applies the chosen method to thyroid, kidney and liver without re-tuning; post_process() function; final figure.

Notebook section	What it does
Conclusion	Guided filter: breast SI 0.22 -> 0.14, 92 % edge kept, about 10-20 ms per image; edge rule not met on thyroid and liver.

3.3 How run_team8.py uses the team's own code

The algorithm is the team's. The despeckling filter (the self-guided filter in `postprocess()` and `guided()`) and the quality metrics (`speckle_index()`, `cnr()`, `gcnr()`, `edge_sharpness()`, `to_uint8()`, plus the helper `lesion_ring()`) all come from the team's existing module `code/post_processing.py`, used **unchanged**. `run_team8.py` is only a demo wrapper: it loads the inputs, calls these functions, and saves numbers and figures.

Where run_team8.py uses your code

run_team8.py line(s)	Team function used	What it does there
79, 89	<code>import post_processing as pp</code>	Line 79 adds <code>code/</code> to the import path; line 89 imports the team's module (as <code>pp</code>). Every <code>pp....</code> call below is team code.
98 (loop 94-102)	<code>pp.postprocess(img)</code>	Despeckles each of the two input images (<code>cyst_lesion</code> , <code>scatterers</code>) with the module's defaults: <code>method='guided'</code> , <code>r = 4</code> , <code>eps = 0.001</code> . Inside, <code>postprocess()</code> calls <code>to_uint8()</code> and <code>guided()</code> .
108, 110	<code>pp.lesion_ring(lesion)</code> , <code>pp.RING</code>	Builds the 10-pixel background ring around the cyst mask; line 110 prints the ring width <code>RING</code> .
114	<code>pp.to_uint8</code>	Inside <code>measure()</code> : converts the <code>[0, 1]</code> image to 0-255 grey levels.
116	<code>pp.speckle_index</code>	Inside <code>measure()</code> : speckle index SI of the cyst image.
117	<code>pp.cnr</code> , <code>pp.gcnr</code> , <code>pp.edge_sharpness</code>	Inside <code>measure()</code> : CNR, gCNR and edge sharpness between lesion and ring.
125	<code>pp.speckle_index</code> , <code>pp.to_uint8</code>	SI before and after for both images (the first rows of the metrics table).
128	<code>measure()</code> (uses the calls above)	All cyst metrics before (<code>m0</code>) and after (<code>m1</code>) the filter.
142-143	<code>pp.postprocess(C["img"], eps=eps)</code> , then <code>measure()</code>	Runs the team's filter again on the cyst with <code>eps = 0.001</code> , <code>0.002</code> , <code>0.005</code> and <code>0.01</code> (<code>r</code> stays 4) and measures each result.

What run_team8.py adds on top (written for this demo, not the team's code)

Line(s)	Added part	Details
37-53	Console copy	Tee class: everything printed goes to the screen and into a text buffer for <code>console_log.txt</code> .
57-65	Helpers	<code>save()</code> stores a figure in <code>output/</code> and remembers its name; <code>banner()</code> prints section titles.
78, 80-88	Empty placeholder 'config' module	<code>post_processing.py</code> does import <code>config</code> only for <code>scan_convert()</code> , which is not used here. If the project's <code>config.py</code> is missing, an empty module named <code>config</code> is registered so the team file imports unchanged. Line 78 stops Python from writing a <code>__pycache__</code> folder into <code>code/</code> .
93-102	Loading the inputs	Reads <code>input/cyst_lesion_image.npz</code> and <code>input/scatterers_image.npz</code> (key 'image'), converts the axes <code>x_img</code> , <code>z_img</code> to mm, times each <code>postprocess()</code> call and prints ranges.
104-107	Lesion mask	Circle of radius 6 mm centred at <code>x = 0</code> , <code>z = 30</code> mm on the image grid (the ring itself is made by the team's <code>lesion_ring()</code>).

Line(s)	Added part	Details
113-117	measure() wrapper	Collects the team's metrics in one dictionary. The contrast in dB (line 116: $50 \times (\text{mean lesion} - \text{mean ring})$ on the uint8 image divided by 255) is computed here in the runner, not by the module.
120-134	Metrics table	Prints before / after / change, the % change of SI and the % of edge sharpness kept.
136-145	eps comparison table	The list of four eps values and the printed SI / gCNR / CNR / edge kept table.
147-227	Figures B1-B6	B1 before/after, B2 cyst edge zoom, B3 difference map, B4 brightness profile, B5 histograms, B6 eps comparison (images and plot).
229-232	Saved result data	output/B_postprocessed_images.npz with both despeckled images (float32).
234-239	Summary	Short summary lines, number of files written and total run time.
241-266	Console log + screenshot	Writes output/console_log.txt and draws output/console_screenshot.png (console_png()).

To see it yourself

Open run_team8.py at the lines above; Cmd/Ctrl+Click a team function (for example postprocess in pp.postprocess) in VS Code and it opens the original file in code/.

3.4 run_team8.py (runner written for this demo)

- Sends all printed text both to the screen and to output/console_log.txt.
- Adds code/ to the import path, registers an empty config placeholder if needed, imports post_processing, runs postprocess() on both inputs, builds the cyst mask (circle $r = 6 \text{ mm}$ at $x = 0, z = 30 \text{ mm}$), computes all metrics with the module's functions on uint8 images, tries four eps values and saves figures B1-B6.
- Finally it writes the console log and a screenshot-style PNG of it.

3.5 Full listings

The listings of post_processing.py and run_team8.py use the real line numbers of the files (a long line that does not fit is continued on the next row, marked with an arrow and no number), so they match the line numbers in Section 3.3.

post_processing.py

```

1  """
2  post_processing.py
3  =====
4
5  T7 image reconstruction + post-processing for the NITK-UsoundSim integration.
6
7      scan_convert(bmode, x_axis, z_axis, ...) -> (image, x_out, z_out)
8      postprocess(image, method="guided", r=4, eps=0.001, gamma=1.0) -> image
9
10 plus the no-reference / lesion quality metrics used to test it
11 (speckle_index, cnr, gcnr, edge_sharpness).
12
13 Inputs
14 -----
15 bmode      : (n_depths, n_lines) B-mode brightness in [0, 1] from T6's
16              bmode_formation() (log-compressed, already on physical axes).
17 x_axis     : (n_lines,) scan-line x-positions [m] (config.get_scanline_positions()).
18 z_axis     : (n_depths,) depths [m] (das_beamform()'s z_axis).
19
20 Outputs
21 -----
22 scan_convert : image (H, W) in [0, 1] on a uniform grid, plus its axes

```

```

23         x_out (W,), z_out (H,) [m]. Row = depth, column = lateral.
24 postprocess : (H, W) float in [0, 1], despeckled.
25
26 Scan conversion (linear array -> rectangular, no fan)
27 -----
28 A linear array's scan lines are parallel and vertical, so the (depth x line)
29 grid is already a Cartesian image; scan conversion is only a resample onto
30 display pixels. Steps: crop depth to z_range (default Z_MIN..Z_MAX, the
31 displayed depth; the RF is recorded to 60 mm only for T5's test), then
32 bilinear interpolation (scipy RegularGridInterpolator) of the B-mode onto
33 the output grid. No polar / sector geometry (that was the old T7 notebook's
34 bug for this probe).
35
36 keep_aspect=True (default): square pixels, the largest grid that fits inside
37 output_size (40 mm x 10 mm into (512, 512) -> 512 x 129 px, 0.078 mm/px).
38 keep_aspect=False: stretch to exactly output_size (non-square pixels).
39 Square pixels matter because postprocess()'s filter radius is in pixels:
40 with 4:1 stretched pixels a 9x9 px window would cover 0.7 mm axially but
41 only 0.18 mm laterally, i.e. smooth speckle in one direction only.
42
43 Post-processing (despeckling)
44 -----
45 The self-guided filter (He et al. 2013) from T7's notebook
46 "262SP009_Post_Image_Processing_v2.ipynb" (Section 4, `guided`), unchanged:
47 the notebook's conclusion picked it out of 10 filters (median, Lee, SRAD, NLM,
48 wavelet, OBNLM, BM3D, guided, bilateral, TV) on real clinical images -- it
49 lowered the speckle index 0.22 -> 0.14 on the breast test set while keeping
50 92 % of lesion-edge sharpness. With guide G = I, window w_k of radius r:
51
52     a_k = cov_k(I, I) / (var_k(I) + eps),    b_k = mean_k(I) - a_k * mean_k(I)
53     q    = mean_w(a) * I + mean_w(b)
54
55 Defaults r = 4, eps = 0.001: the notebook's tuned fixed value (its auto-eps
56 fit is weak, r = 0.54, and was fitted on breast noise levels only). The
57 image is quantised to uint8 exactly as in the notebook, filtered, and
58 returned as float in [0, 1]. method="none" skips the filter.
59 Grey-level mapping: out = filtered ** gamma (gamma = 1.0, the default, leaves
60 it unchanged; < 1 brightens dark regions, > 1 darkens them).
61
62 TIME-AXIS RULE (README Section 6) -- not applicable here: no delay-to-sample
63 lookup happens in this module. Its inputs already carry physical axes
64 (das_beamform() did the np.interp lookups on the real t_axis); resampling
65 uses those axes, never sample indices.
66
67 Assumptions / limitations
68 -----
69 - Bilinear resampling of log-compressed data (standard display practice);
70   upsampling 64 lines to ~129 px adds no lateral resolution.
71 - Filter parameters were tuned on real clinical images (breast), not on this
72   simulator; they are reused, not re-tuned.
73 - Metric definitions are the notebook's (SI 7x7, CNR/gCNR ring of 10 px),
74   so pixel-based sizes depend on the output pixel size.
75 """
76
77 import cv2
78 import numpy as np
79 from scipy.interpolate import RegularGridInterpolator
80
81 import config
82
83
84 # -----
85 # Scan conversion
86 # -----
87 def scan_convert(bmode, x_axis, z_axis, output_size=None, z_range=None, keep_aspect=True):
88     """Resample the (depth x line) B-mode onto display pixels; see module docstring."""
89     bmode = np.asarray(bmode, dtype=float)
90     x_axis = np.asarray(x_axis, dtype=float)
91     z_axis = np.asarray(z_axis, dtype=float)
92     if bmode.shape != (z_axis.size, x_axis.size):
93         raise ValueError(f"bmode shape {bmode.shape} != (len(z_axis), len(x_axis)) = {(z_axis.size, x_axis.size)}")
94     h_max, w_max = config.OUTPUT_SIZE if output_size is None else output_size
95     z0, z1 = (config.Z_MIN, config.Z_MAX) if z_range is None else z_range
96     z0, z1 = max(z0, z_axis[0]), min(z1, z_axis[-1])
97     x0, x1 = x_axis[0], x_axis[-1]
98

```

```

99     if keep_aspect:
100         pixel = max((z1 - z0) / (h_max - 1), (x1 - x0) / (w_max - 1))
101         h = int(np.floor((z1 - z0) / pixel + 1e-9)) + 1
102         w = int(np.floor((x1 - x0) / pixel + 1e-9)) + 1
103     else:
104         h, w = h_max, w_max
105     z_out = np.linspace(z0, z1, h)
106     x_out = np.linspace(x0, x1, w)
107
108     interp = RegularGridInterpolator((z_axis, x_axis), bmode, method="linear")
109     zz, xx = np.meshgrid(z_out, x_out, indexing="ij")
110     image = interp(np.stack([zz.ravel(), xx.ravel()], axis=1)).reshape(h, w)
111     return np.clip(image, 0.0, 1.0), x_out, z_out
112
113
114 # -----
115 # Despeckling: T7 notebook's guided filter (Section 4), unchanged
116 # -----
117 def _finish(out, img, scan):
118     out = np.clip(np.round(out), 0, 255).astype(np.uint8)
119     out[~scan] = img[~scan]
120     return out
121
122
123 def guided(img, scan, r=4, eps=0.01, guide=None):
124     """Notebook's guided filter: uint8 img, bool scan mask -> uint8 (guide=None -> self-guided)."""
125     I = img.astype(np.float32) / 255; G = I if guide is None else guide.astype(np.float32) / 255
126     box = lambda x: cv2.blur(x, (2 * r + 1, 2 * r + 1))
127     mI, mG = box(I), box(G); cov = box(G * I) - mG * mI; var = box(G * G) - mG * mG
128     a = cov / (var + eps); b = mI - a * mG
129     return _finish(255 * (box(a) * G + box(b)), img, scan)
130
131
132 def to_uint8(image):
133     """[0, 1] float -> uint8 grey levels (the notebook's working format)."""
134     return np.clip(np.round(255 * np.asarray(image, dtype=float)), 0, 255).astype(np.uint8)
135
136
137 def postprocess(image, method="guided", r=4, eps=0.001, gamma=1.0):
138     """Despeckle a scan-converted [0, 1] image, then gamma-map; returns float [0, 1]. See module d
139     ocstring."""
140     image = np.asarray(image, dtype=float)
141     if image.ndim != 2:
142         raise ValueError("image must be 2-D")
143     if gamma <= 0:
144         raise ValueError("gamma must be positive")
145     if method == "none":
146         out = image.copy()
147     elif method == "guided":
148         img = to_uint8(image)
149         out = guided(img, np.ones(img.shape, bool), r=r, eps=eps).astype(float) / 255
150     else:
151         raise ValueError(f"method must be 'guided' or 'none', got {method!r}")
152     return out if gamma == 1.0 else out ** gamma
153
154 # -----
155 # Quality metrics (notebook Section 3 definitions, on uint8 images)
156 # -----
157 RING = 10
158 _kernel = lambda k: cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))
159
160
161 def speckle_index(img, scan_mask=None, k=7):
162     """Mean of local std / local mean (k x k) inside the scan area; lower = less speckle."""
163     f = img.astype(np.float32)
164     scan_mask = np.ones(f.shape, bool) if scan_mask is None else scan_mask
165     mu = cv2.blur(f, (k, k)); sd = np.sqrt(np.maximum(cv2.blur(f * f, (k, k)) - mu * mu, 0))
166     valid = (cv2.erode(scan_mask.astype(np.uint8), _kernel(k)) > 0) & (mu > 1)
167     return float(np.mean(sd[valid] / mu[valid]))
168
169
170 def lesion_ring(lesion_mask):
171     """Background ring: lesion dilated by RING px, minus the lesion."""
172     return (cv2.dilate(lesion_mask.astype(np.uint8), _kernel(2 * RING + 1)) > 0) & ~lesion_mask
173

```

```

174
175 def cnr(img, lesion_mask):
176     """|mu_lesion - mu_ring| / sqrt(var_lesion + var_ring); ring = lesion dilated by RING px."""
177     a, b = img[lesion_mask].astype(np.float64), img[lesion_ring(lesion_mask)].astype(np.float64)
178     return abs(a.mean() - b.mean()) / np.sqrt(a.var() + b.var() + 1e-9)
179
180
181 def gcnr(img, lesion_mask):
182     """1 - overlap of lesion and ring grey-level histograms (0 = identical, 1 = separable)."""
183     a, b = img[lesion_mask], img[lesion_ring(lesion_mask)]
184     ha = np.bincount(a, minlength=256) / a.size; hb = np.bincount(b, minlength=256) / b.size
185     return 1 - np.minimum(ha, hb).sum()
186
187
188 def edge_sharpness(img, lesion_mask):
189     """Mean Sobel gradient magnitude on a 3-px band across the lesion border."""
190     f = img.astype(np.float32)
191     g = np.hypot(cv2.Sobel(f, cv2.CV_32F, 1, 0), cv2.Sobel(f, cv2.CV_32F, 0, 1))
192     m = lesion_mask.astype(np.uint8)
193     band = cv2.dilate(m, _kernel(3)) != cv2.erode(m, _kernel(3))
194     return float(g[band].mean())

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 1

```

1 # Colab setup (kagglehub, OpenCV, scikit-image, SciPy and matplotlib are already in Colab; bm3d al
2   ↳ so installs PyWavelets)
3 %pip install -q kagglehub bm3d

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 2

```

1 import os, json, glob, random, shutil, zipfile, io, time
2 from pathlib import Path
3 import numpy as np, cv2, requests, kagglehub
4 import matplotlib.pyplot as plt
5
6 SEED = 0
7 random.seed(SEED); np.random.seed(SEED)
8 IN_COLAB = 'google.colab' in str(get_ipython())
9 DATA = Path('/content/data' if IN_COLAB else 'data'); DATA.mkdir(parents=True, exist_ok=True)
10 N_PER_ORGAN, MAX_SIDE = 20, 512
11 ORGANS = ['breast', 'thyroid', 'kidney', 'liver']

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 4

```

1 def fetch(url, dest, tries=3):
2     dest, part = Path(dest), Path(str(dest) + '.part')
3     for t in range(tries):
4         if dest.exists(): break
5         try:
6             ↳ complete
7             with requests.get(url, stream=True, timeout=(30, 60)) as r:
8                 r.raise_for_status()
9                 with open(part, 'wb') as fh:
10                     for chunk in r.iter_content(1 << 20): fh.write(chunk)
11                 part.rename(dest)
12             except requests.RequestException as e:
13                 print(f'download attempt {t + 1} failed: {e}')
14             if not dest.exists(): raise RuntimeError(f'could not download {url}')
15             return dest
16
17 def unzip_once(zpath, out):
18     if not Path(out).exists(): shutil.unpack_archive(str(zpath), str(out))
19     return Path(out)
20
21 def to_gray(img):
22     return img if img.ndim == 2 else cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
23
24 def resize_max(img, mask=None, max_side=MAX_SIDE):
25     s = max_side / max(img.shape[:2])
26     if s >= 1: return img, mask
27     size = (round(img.shape[1] * s), round(img.shape[0] * s))
28     img = cv2.resize(img, size, interpolation=cv2.INTER_AREA)
29     if mask is not None:

```

```

30     mask = cv2.resize(mask.astype(np.uint8), size, interpolation=cv2.INTER_NEAREST).astype(boo
31     ↪ l)
32     return img, mask
33
34 def polygon_mask(points, shape):
35     m = np.zeros(shape, np.uint8)
36     polys = [points] if np.ndim(points[0]) == 1 else points      # one polygon or a list of them
37     for p in polys: cv2.fillPoly(m, [np.round(np.array(p)).astype(np.int32)], 1)
38     return m.astype(bool)

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 6

```

1  UPLOAD_KIDNEY = False
2  KIDNEY_IMG_DIR = DATA / 'kidney' / 'images'
3  if UPLOAD_KIDNEY and IN_COLAB:
4      from google.colab import files
5      for fn in files.upload(): shutil.unpack_archive(fn, KIDNEY_IMG_DIR)

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 7

```

1  def list_breast():
2      root = Path(kagglehub.dataset_download('aryashah2k/breast-ultrasound-images-dataset')) / 'Data
3      ↪ set_BUSI_with_GT'
4      items = []
5      for f in sorted(glob.glob(str(root / '*' / '*.png'))):          # benign/malignant/norm
6      ↪ al images
7          masks = sorted(glob.glob(f[:-4] + '_mask*.png'))
8          items.append((f, lambda sh, m=masks: np.any([cv2.imread(x, 0) > 127 for x in m], axis=0)))
9      return items
10
11 def list_thyroid():
12     root = Path(kagglehub.dataset_download('tjahan/tn3k-thyroid-nodule-region-segmentation-dataset
13     ↪ '))
14     return [(f, lambda sh, f=f: cv2.imread(f.replace('test-image', 'test-mask'), 0) > 127)
15             for f in sorted(glob.glob(str(root / 'test-image' / '*.jpg')))]
16
17 def list_liver():
18     items = []
19     for cls in ['Benign', 'Malignant']:
20         z = fetch(f'https://zenodo.org/api/records/7272660/files/{cls}.zip/content', DATA / f'live
21     ↪ r_{cls}.zip')
22         root = unzip_once(z, DATA / 'liver' / cls) / cls
23         for f in sorted(glob.glob(str(root / 'image' / '*.jpg'))):
24             js = root / 'segmentation' / 'mass' / (Path(f).stem + '.json')
25             items.append((f, lambda sh, js=js: polygon_mask(json.load(open(js)), sh)))
26     return items
27
28 def list_kidney():
29     imgs = sorted(p for p in KIDNEY_IMG_DIR.rglob('*') if p.suffix.lower() in ('.png', '.jpg', '.j
30     ↪ peg'))
31     if imgs:          # Open Kidney + public capsul
32     ↪ e masks
33         z = fetch('https://codecademy.github.com/rsingla92/kidneyUS/zip/refs/heads/main', DATA / 'ki
34     ↪ dneyUS.zip')
35         mdir = unzip_once(z, DATA / 'kidneyUS') / 'kidneyUS-main' / 'labels' / 'reviewed_masks_1'
36     ↪ / 'capsule'
37         return [(str(p), lambda sh, m=mdir / (p.stem + '.png'): cv2.imread(str(m), 0) > 0)
38                 for p in imgs if (mdir / (p.stem + '.png')).exists()]
39     print('kidney: Open Kidney images not found -> using fallback Kaggle renal-data (no masks)')
40     root = Path(kagglehub.dataset_download('zaynebnouiri/renal-data'))
41     return [(f, None) for f in sorted(glob.glob(str(root / 'images' / '*' / '*.jpg')))]
42
43 LISTERS = {'breast': list_breast, 'thyroid': list_thyroid, 'kidney': list_kidney, 'liver': list_li
44     ↪ ver}
45
46 def load_dataset(organ, n=N_PER_ORGAN, seed=SEED):
47     items = LISTERS[organ]()
48     if any(m is not None for _, m in items):          # prefer images with a non-em
49     ↪ pty mask
50         items = [it for it in items if it[1] is not None]
51         rng = np.random.default_rng(seed)
52         data = []
53         for i in rng.permutation(len(items)):
54             f, get_mask = items[i]

```

```

55     img = to_gray(cv2.imread(f))
56     mask = get_mask(img.shape) if get_mask else None
57     if mask is not None and (mask.shape != img.shape or not mask.any()): continue
58     img, mask = resize_max(img, mask)
59     data.append({'image': img, 'mask': mask, 'name': f'{organ}/{Path(f).name}'})
60     if len(data) == n: break
61     return data

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 8

```

1  datasets = {}
2  for organ in ORGANS:
3      try:
4          datasets[organ] = load_dataset(organ)
5      except Exception as e:
6          print(f'{organ}: FAILED -> {type(e).__name__}: {e}'); datasets[organ] = []
7
8  for organ, d in datasets.items():
9      n_mask = sum(x['mask'] is not None for x in d)
10     sizes = [x['image'].shape for x in d]
11     print(f'{organ:8s} images: {len(d):2d} masks: {n_mask:2d} '
12           f'size range: {min(sizes) if sizes else "-"} .. {max(sizes) if sizes else "-"}')

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 9

```

1  fig, axes = plt.subplots(4, 4, figsize=(14, 13))
2  for r, organ in enumerate(ORGANS):
3      for c in range(4):
4          ax = axes[r, c]; ax.axis('off')
5          if c >= len(datasets[organ]): continue
6          s = datasets[organ][c]
7          ax.imshow(s['image'], cmap='gray', vmin=0, vmax=255)
8          if s['mask'] is not None: ax.contour(s['mask'], [0.5], colors='r', linewidths=1)
9          ax.set_title(s['name'], fontsize=8)
10     axes[r, 0].text(-0.08, 0.5, organ, transform=axes[r, 0].transAxes, rotation=90,
11                    va='center', ha='right', fontsize=13)
12 plt.tight_layout(); plt.show()

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 11

```

1  def auto_mask(img, t=3, min_frac=0.01):
2      m = (cv2.blur(img, (9, 9)) > t).astype(np.uint8)
3      m = cv2.morphologyEx(m, cv2.MORPH_OPEN, np.ones((7, 7), np.uint8))
4      n, lab, st, _ = cv2.connectedComponentsWithStats(m)
5      big = [i for i in range(1, n) if st[i, cv2.CC_STAT_AREA] > min_frac * img.size]
6      if not big: return np.ones(img.shape, bool)
7      hull = cv2.convexHull(cv2.findNonZero(np.isin(lab, big).astype(np.uint8)))
8      out = np.zeros(img.shape, np.uint8); cv2.fillPoly(out, [hull], 1)
9      return cv2.erode(out, np.ones((5, 5), np.uint8)) > 0
10
11 def find_annotations(img, scan=None, lap_t=150, int_t=180, ratio_t=0.5):
12     f = img.astype(np.float32)
13     diff = f - cv2.medianBlur(img, 15) # brightness above local bac
14     kground
15     lap = np.abs(cv2.Laplacian(f, cv2.CV_32F, ksize=1)) # edge sharpness
16     cand = ((diff > 40) & (img > 120)).astype(np.uint8)
17     seed = (lap > lap_t) & (diff > 60) & (img > int_t)
18     if scan is not None: seed &= cv2.erode(scan.astype(np.uint8), np.ones((15, 15), np.uint8)) > 0
19     near = cv2.dilate(seed.astype(np.uint8), np.ones((3, 3), np.uint8)) > 0
20     n, lab = cv2.connectedComponents(cand, connectivity=8)
21     ratio = np.bincount(lab[near], minlength=n) / np.maximum(np.bincount(lab.ravel(), minlength=n)
22     , 1)
23     keep = np.flatnonzero(ratio >= ratio_t); keep = keep[keep > 0]
24     return cv2.dilate(np.isin(lab, keep).astype(np.uint8), np.ones((3, 3), np.uint8)) > 0
25
26 def clean_annotations(img, scan=None):
27     marks = find_annotations(img, scan)
28     return cv2.inpaint(img, marks.astype(np.uint8), 3, cv2.INPAINT_TELEA), marks
29
30 def normalize(img, mask, lo=0.5, hi=99.5):
31     a, b = np.percentile(img[mask], [lo, hi])
32     out = np.clip((img.astype(np.float32) - a) * 255 / max(b - a, 1), 0, 255).astype(np.uint8)
33     out[~mask] = 0

```

```

34     return out
35
36 def prepare(img):
37     scan = auto_mask(img)
38     clean, _ = clean_annotations(img, scan)
39     return normalize(clean, scan), scan

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 12

```

1  for organ in ORGANS:
2      for s in datasets[organ]:
3          s['clean'], s['scan'] = prepare(s['image'])
4          s['marks'] = find_annotations(s['image'], s['scan'])
5
6  print('organ    removed px % (mean / max)    lesion px removed %    lesion outside scan %    scan are
7  ↳ a %')
8  for organ in ORGANS:
9      d = datasets[organ]
10     rem = [100 * s['marks'].mean() for s in d]
11     les = [100 * (s['marks'] & s['mask']).sum() / s['mask'].sum() for s in d if s['mask'] is not N
12     ↳ one]
13     out = [100 * (s['mask'] & ~s['scan']).sum() / s['mask'].sum() for s in d if s['mask'] is not N
14     ↳ one]
15     area = [100 * s['scan'].mean() for s in d]
16     fmt = lambda v: f'{np.mean(v):6.2f} / {np.max(v):5.2f}' if v else '    n/a    '
17     print(f'{organ:8s} {fmt(rem):>22s} {fmt(les):>18s} {fmt(out):>18s} {np.mean(area):6.1f}'
18     ↳ )

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 13

```

1  show = [s for organ in ORGANS for s in sorted(datasets[organ], key=lambda s: -s['marks'].sum())[:2
2  ↳ ]]
3  fig, axes = plt.subplots(len(show), 4, figsize=(13, 3 * len(show)))
4  for row, s in zip(axes, show):
5      over = np.dstack([s['image']] * 3); over[s['marks']] = (255, 0, 0)
6      for ax, im, t in zip(row, [s['image'], over, s['clean'], s['scan'] * 255],
7      ↳ [s['name'], f'removed: {100 * s['marks'].mean():.2f}% px", 'cleaned + nor
8      ↳ malized', 'scan mask']):
9          ax.imshow(im, cmap='gray', vmin=0, vmax=255); ax.set_title(t, fontsize=9); ax.axis('off')
10 plt.tight_layout(); plt.show()

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 15

```

1  RING, ROI_SIZE = 10, 31
2  kernel = lambda k: cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))
3
4  def timed(fn, *args, **kw):
5      t0 = time.perf_counter(); out = fn(*args, **kw)
6      return out, 1000 * (time.perf_counter() - t0)
7
8  def enl(img, roi):
9      v = img[roi].astype(np.float64)
10     return v.mean() ** 2 / max(v.var(), 1e-9)
11
12 def speckle_index(img, scan_mask, k=7):
13     f = img.astype(np.float32)
14     mu = cv2.blur(f, (k, k)); sd = np.sqrt(np.maximum(cv2.blur(f * f, (k, k)) - mu * mu, 0))
15     valid = (cv2.erode(scan_mask.astype(np.uint8), kernel(k)) > 0) & (mu > 1)
16     return float(np.mean(sd[valid] / mu[valid]))
17
18 def lesion_ring(lesion_mask, scan_mask=None):
19     ring = (cv2.dilate(lesion_mask.astype(np.uint8), kernel(2 * RING + 1)) > 0) & ~lesion_mask
20     return ring & scan_mask if scan_mask is not None else ring
21
22 def cnr(img, lesion_mask, scan_mask=None):
23     a, b = img[lesion_mask].astype(np.float64), img[lesion_ring(lesion_mask, scan_mask)].astype(np
24     ↳ .float64)
25     return abs(a.mean() - b.mean()) / np.sqrt(a.var() + b.var() + 1e-9)
26
27 def gcnr(img, lesion_mask, scan_mask=None):
28     a, b = img[lesion_mask], img[lesion_ring(lesion_mask, scan_mask)]
29     ha = np.bincount(a, minlength=256) / a.size; hb = np.bincount(b, minlength=256) / b.size
30     return 1 - np.minimum(ha, hb).sum()

```

```

31
32 def edge_sharpness(img, lesion_mask):
33     f = img.astype(np.float32)
34     g = np.hypot(cv2.Sobel(f, cv2.CV_32F, 1, 0), cv2.Sobel(f, cv2.CV_32F, 0, 1))
35     m = lesion_mask.astype(np.uint8)
36     band = cv2.dilate(m, kernel(3)) != cv2.erode(m, kernel(3))
37     return float(g[band].mean())
38
39 def auto_background_roi(img, scan_mask, lesion_mask=None, w=ROI_SIZE):
40     f = img.astype(np.float32)
41     local_stats = lambda x: (cv2.blur(x, (w, w)), np.sqrt(np.maximum(cv2.blur(x * x, (w, w)) - cv2
42     ↪ .blur(x, (w, w)) ** 2, 0)))
43     mu, _ = local_stats(f)
44     _, structure = local_stats(cv2.blur(f, (9, 9))) # variation left after smoot
45     ↪ hing = anatomy
46     base = cv2.erode(scan_mask.astype(np.uint8), np.ones((w + 2, w + 2), np.uint8)) > 0
47     base[:, w, :] = base[:, -w, :] = base[:, :w] = base[:, -w:] = False # whole window inside the ima
48     ↪ ge
49     flat = np.zeros(img.shape, np.float32) # pixel within 1 grey level
50     ↪ of right/lower neighbour
51     flat[:, -1, :-1] = (np.abs(f[:, -1, :-1] - f[:, :-1, :-1]) <= 1) & (np.abs(f[:, -1, :-1] - f[:, :-1, 1:]) <
52     ↪ = 1)
53     base &= (mu > np.percentile(f[scan_mask], 25)) & (cv2.blur(flat, (w, w)) < 0.5) # no shadow,
54     ↪ no flat padding
55     for margin in (2 * w, w, w // 2, 0): # relax lesion distance if n
56     ↪ othing is left
57         ok = base.copy()
58         if lesion_mask is not None and margin: ok &= cv2.dilate(lesion_mask.astype(np.uint8), kern
59     ↪ el(2 * margin + 1)) == 0
60         if ok.any(): break
61     y, x = np.unravel_index(np.argmin(np.where(ok, structure / np.maximum(mu, 1), np.inf)), mu.sha
62     ↪ pe)
63     roi = np.zeros(img.shape, bool); roi[y - w // 2: y + w // 2 + 1, x - w // 2: x + w // 2 + 1] =
64     ↪ True
65     return roi
66
67 def metrics(img, s, ms=np.nan):
68     r = {'ENL': enl(img, s['roi']), 'SI': speckle_index(img, s['scan']), 'ms': ms}
69     if s['mask'] is not None:
70         r.update(CNR=cnr(img, s['mask'], s['scan']), gCNR=gcnr(img, s['mask'], s['scan']),
71         Edge=edge_sharpness(img, s['mask']))
72     return r

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 16

```

1  METRICS = ['ENL', 'SI', 'CNR', 'gCNR', 'Edge', 'ms']
2
3  def table(rows, title):
4      print(title); print(f"{'organ':8s} {'input':10s}" + ''.join(f'{m:>16s}' for m in METRICS))
5      for (organ, name), res in rows.items():
6          cells = []
7          for m in METRICS:
8              v = np.array([r[m] for r in res if m in r and np.isfinite(r[m])])
9              cells.append(f'{v.mean():7.2f} ± {v.std():5.2f}' if v.size else 'n/a')
10             print(f"{'organ':8s} {'name':10s}" + ''.join(f'{c:>16s}' for c in cells))
11
12  baseline = {}
13  for organ in ORGANS:
14      for s in datasets[organ]:
15          (s['clean'], s['scan']), ms = timed(prepare, s['image'])
16          s['roi'] = auto_background_roi(s['image'], s['scan'], s['mask'])
17          s['base'] = metrics(s['clean'], s, ms)
18          baseline.setdefault((organ, 'original'), []).append(metrics(s['image'], s))
19          baseline.setdefault((organ, 'prepared'), []).append(s['base'])
20  table(baseline, 'Baseline (no despeckling). ms = prepare() time per image. Mean ± std over 20 imag
21  ↪ es.')

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 17

```

1  fig, axes = plt.subplots(1, 4, figsize=(16, 4.5))
2  for ax, organ in zip(axes, ORGANS):
3      s = next((s for s in datasets[organ] if s['mask'] is not None), datasets[organ][0])
4      ax.imshow(s['clean'], cmap='gray', vmin=0, vmax=255); ax.axis('off'); ax.set_title(s['name'],
5      ↪ fontsize=9)

```

```

6     ax.contour(s['roi'], [0.5], colors='y')
7     if s['mask'] is not None:
8         ax.contour(lesion_ring(s['mask'], s['scan']), [0.5], colors='c'); ax.contour(s['mask'], [0
9         ↪ .5], colors='r')
10 fig.suptitle('red = lesion, cyan = background ring, yellow = ENL ROI'); plt.tight_layout(); plt.sh
11 ↪ ow()

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 19

```

1 from skimage.restoration import denoise_nl_means, denoise_wavelet, denoise_tv_chambolle
2 import bm3d as bm3d_lib
3
4 def finish(out, img, scan):
5     out = np.clip(np.round(out), 0, 255).astype(np.uint8)
6     out[~scan] = img[~scan]
7     return out
8
9 def noise_sigma(f, scan, k=7):                                # speckle is correlated, s
10     ↪ o wavelet-MAD
11     mu = cv2.blur(f, (k, k))                                # (estimate_sigma) underes
12     ↪ timates it;
13     sd = np.sqrt(np.maximum(cv2.blur(f * f, (k, k)) - mu * mu, 0))    # use the median local std
14     ↪ instead
15     return float(np.median(sd[cv2.erode(scan.astype(np.uint8), kernel(k)) > 0]))
16
17 def median(img, scan, k=5):
18     return finish(cv2.medianBlur(img, k), img, scan)
19
20 def lee(img, scan, k=7):
21     f = img.astype(np.float32)
22     mu = cv2.blur(f, (k, k)); var = np.maximum(cv2.blur(f * f, (k, k)) - mu * mu, 0)
23     ci2 = var / np.maximum(mu * mu, 1)
24     cu2 = np.median(ci2[scan & (mu > 10)])                    # noise CoV2 = typical loc
25     ↪ al CoV2
26     w = np.clip(1 - cu2 / np.maximum(ci2, 1e-6), 0, 1)
27     return finish(mu + w * (f - mu), img, scan)
28
29 def srad(img, scan, n_iter=50, dt=0.1):
30     I = img.astype(np.float32) / 255 + 1e-3
31     for _ in range(n_iter):
32         P = np.pad(I, 1, mode='edge')
33         dN, dS, dW, dE = P[:-2, 1:-1] - I, P[2:, 1:-1] - I, P[1:-1, :-2] - I, P[1:-1, 2:] - I
34         g2 = (dN ** 2 + dS ** 2 + dW ** 2 + dE ** 2) / I ** 2
35         lap = (dN + dS + dW + dE) / I
36         q2 = np.maximum((0.5 * g2 - lap ** 2 / 16) / (1 + lap / 4) ** 2, 0)
37         q02 = np.median(q2[scan])                                # speckle scale from the
38     ↪ image itself
39     c = np.clip(1 / (1 + (q2 - q02) / (q02 * (1 + q02) + 1e-12)), 0, 1)
40     C = np.pad(c, 1, mode='edge')
41     I = I + dt / 4 * (C[2:, 1:-1] * dS + c * dN + C[1:-1, 2:] * dE + c * dW)
42     return finish(255 * (I - 1e-3), img, scan)
43
44 def nlm(img, scan, h=0.8, patch=5, dist=6):
45     f = img.astype(np.float32) / 255
46     sigma = noise_sigma(f, scan)
47     out = denoise_nl_means(f, h=h * sigma, sigma=sigma, patch_size=patch, patch_distance=dist, fas
48     ↪ t_mode=True)
49     return finish(255 * out, img, scan)
50
51 def wavelet(img, scan, scale=1.0, wavelet_name='db4', levels=4):
52     L = np.log1p(img.astype(np.float32)); m = L.max()
53     sigma = scale * noise_sigma(L / m, scan)
54     out = denoise_wavelet(L / m, sigma=sigma, wavelet=wavelet_name, mode='soft', method='BayesShri
55     ↪ nk',
56                             wavelet_levels=levels)
57     return finish(np.expml(out * m), img, scan)
58
59 def obnlm(img, scan, h=1.0, patch=5, search=11):
60     f = img.astype(np.float32) + 1; H, W = f.shape; r = search // 2
61     pad = cv2.copyMakeBorder(f, r, r, r, r, cv2.BORDER_REFLECT)
62     num, den = np.zeros_like(f), np.zeros_like(f)
63     mu = cv2.blur(f, (patch, patch)); var = np.maximum(cv2.blur(f * f, (patch, patch)) - mu * mu,
64     ↪ 0)
65     h2 = h * 2 * np.median((var / mu)[scan])                    # Pearson distance of two p
66     ↪ ure-speckle patches ~ 2·var/mean
67     for dy in range(-r, r + 1):

```

```

68         for dx in range(-r, r + 1):
69             g = pad[r + dy: r + dy + H, r + dx: r + dx + W]
70             d = cv2.blur((f - g) ** 2 / g, (patch, patch))           # Pearson distance, ave
71         ← raged over patch
72             w = np.exp(-d / h2)
73             num += w * g; den += w
74         return finish(num / den - 1, img, scan)
75
76 def bm3d_log(img, scan, scale=1.0):
77     L = np.loglp(img.astype(np.float32)); m = L.max()
78     out = bm3d_lib.bm3d(L / m, sigma_psd=scale * noise_sigma(L / m, scan))
79     return finish(np.expml(out * m), img, scan)
80
81 def guided(img, scan, r=4, eps=0.01, guide=None):                     # guide=None -> self-guided
82     I = img.astype(np.float32) / 255; G = I if guide is None else guide.astype(np.float32) / 255
83     box = lambda x: cv2.blur(x, (2 * r + 1, 2 * r + 1))
84     mI, mG = box(I), box(G); cov = box(G * I) - mG * mI; var = box(G * G) - mG * mG
85     a = cov / (var + eps); b = mI - a * mG
86     return finish(255 * (box(a) * G + box(b)), img, scan)
87
88 def bilateral(img, scan, sigma_color=40, sigma_space=5, d=9):
89     return finish(cv2.bilateralFilter(img, d, sigma_color, sigma_space), img, scan)
90
91 def tv(img, scan, weight=0.1):
92     return finish(255 * denoise_tv_chambolle(img.astype(np.float32) / 255, weight=weight), img, sc
93     ← an)
94
95 FILTERS = {'Median': median, 'Lee': lee, 'SRAD': srاد, 'NLM': nlm, 'Wavelet': wavelet, 'OBNLM': ob
96     ← nlm,
97     'BM3D': bm3d_log, 'Guided': guided, 'Bilateral': bilateral, 'TV': tv}
98 GRIDS = {'Median': {'k': [3, 5, 7, 9]}, 'Lee': {'k': [5, 7, 9, 11]}, 'SRAD': {'n_iter': [20, 50, 1
99     ← 00, 200]},
100     'NLM': {'h': [0.6, 0.8, 1.0, 1.3]}, 'Wavelet': {'scale': [0.5, 1.0, 2.0, 3.0]},
101     'OBNLM': {'h': [0.5, 1.0, 2.0, 4.0]}, 'BM3D': {'scale': [0.5, 1.0, 2.0, 3.0]},
102     'Guided': {'eps': [0.001, 0.005, 0.01, 0.03]}, 'Bilateral': {'sigma_color': [20, 40, 60,
103     ← 90]},
104     'TV': {'weight': [0.05, 0.1, 0.2, 0.3]}}

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 20

```

1  s = next(s for s in datasets['breast'] if s['mask'] is not None)
2  timed(bm3d_log, s['clean'], s['scan'])                             # warm-up (first call imp
3  ← orts/compiles)
4  outs, rows = {}, {'breast', 'prepared': [s['base']]}
5  for name, fn in FILTERS.items():
6      outs[name], ms = timed(fn, s['clean'], s['scan'])
7      rows[('breast', name)] = [metrics(outs[name], s, ms)]
8  table(rows, f"{s['name']} {s['clean'].shape}, default params (single image, no tuning)")
9  slow = [n for n in FILTERS if rows[('breast', n)][0]['ms'] > 5000]
10 print('methods over 5 s per image:', slow or 'none')
11
12 fig, axes = plt.subplots(len(FILTERS), 2, figsize=(12, 5 * len(FILTERS))) # one row per filter:
13     ← before | after
14 for row, (name, out) in zip(axes, outs.items()):
15     r = rows[('breast', name)][0]
16     for ax, im, t in zip(row, [s['clean'], out], ['before (prepared input)', f"after: {name} ({r[
17     ← 'ms']:.0f} ms)"]):
18         ax.imshow(im, cmap='gray', vmin=0, vmax=255); ax.contour(s['mask'], [0.5], colors='r', lin
19     ← ewidths=0.6)
20         ax.set_title(t, fontsize=12); ax.set_xticks([]); ax.set_yticks([])
21         row[1].set_xlabel(f"ENL {r['ENL']:.0f} SI {r['SI']:.2f} gCNR {r['gCNR']:.2f} Edge {r['Ed
22     ← ge']:.0f}", fontsize=10)
23 fig.suptitle(f"{s['name']}: before / after each filter (default params, red = lesion)", y=1.0)
24 plt.tight_layout(); plt.show()
25
26 zoom = [x for x in datasets['breast'] if x['mask'] is not None][:3] # lesion close-ups, 3 brea
27     ← st images
28 fig, axes = plt.subplots(len(zoom), 11, figsize=(22, 2.3 * len(zoom)))
29 for row, z in zip(axes, zoom):
30     ys, xs = np.nonzero(z['mask']); pad = 40
31     y0, y1, x0, x1 = max(ys.min() - pad, 0), ys.max() + pad, max(xs.min() - pad, 0), xs.max() + pa
32     ← d
33     ims = [('input', z['clean'])] + [(n, fn(z['clean'], z['scan'])) for n, fn in FILTERS.items()]
34     for ax, (name, im) in zip(row, zip(ims)):
35         ax.imshow(im[y0:y1, x0:x1], cmap='gray', vmin=0, vmax=255); ax.axis('off')
36         ax.contour(z['mask'][y0:y1, x0:x1], [0.5], colors='r', linewidths=0.6); ax.set_title(name,

```

```

37     ↪ fontsize=9)
38     row[0].text(-0.1, 0.5, z['name'].split('/')[1], transform=row[0].transAxes, rotation=90, va='c
39     ↪ enter', ha='right', fontsize=8)
40 fig.suptitle('lesion close-ups (lesion box + 40 px)'); plt.tight_layout(); plt.show()

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 22

```

1  EDGE_MIN = 0.90
2  breast = [s for s in datasets['breast'] if s['mask'] is not None]
3  order = np.random.default_rng(SEED).permutation(len(breast))
4  tune_set, test_set = [breast[i] for i in order[:10]], [breast[i] for i in order[10:20]]
5
6  def run_method(method, params, data):
7      res = []
8      for s in data:
9          out, ms = timed(FILTERS[method], s['clean'], s['scan'], **params)
10         r = metrics(out, s, ms); r['EdgeRatio'] = r['Edge'] / s['base']['Edge']
11         res.append(r)
12     return res
13
14  def tune(method, grid, tune_set):
15      (p, values), = grid.items()
16      log = [] # (value, mean CNR, mean edge ra
17      ↪ tio)
18      for v in values:
19          res = run_method(method, {p: v}, tune_set)
20          log.append((v, np.mean([r['CNR'] for r in res]), np.mean([r['EdgeRatio'] for r in res])))
21      ok = [l for l in log if l[2] >= EDGE_MIN]
22      best = max(ok, key=lambda l: l[1]) if ok else max(log, key=lambda l: l[2])
23      return {p: best[0]}, bool(ok), log
24
25  def evaluate(method, params, test_set):
26      res = run_method(method, params, test_set)
27      return {m: np.array([r[m] for r in res]) for m in res[0]}
28
29  tuned, results = {}, {}
30  for name in FILTERS:
31      params, ok, log = tune(name, GRIDS[name], tune_set)
32      tuned[name] = (params, ok)
33      print(f"{name:9s} " + ' '.join(f"{v}: CNR {c:.3f} edge {e:.2f}" for v, c, e in log) + f" ->
34      ↪ {params} rule {'ok' if ok else 'FAILED'}")
35      results[name] = evaluate(name, params, test_set)

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 23

```

1  cols = ['ENL', 'SI', 'CNR', 'gCNR', 'EdgeRatio', 'ms']
2  base = {m: np.array([s['base'][m] if m != 'EdgeRatio' else 1.0 for s in test_set]) for m in cols[:
3  ↪ -1]}
4  rank = sorted(FILTERS, key=lambda n: -results[n]['gCNR'].mean())
5  print(f"Test set (10 breast images), mean ± std, sorted by gCNR. Edge rule: tune-set edge ratio ≥
6  ↪ {EDGE_MIN}.")
7  print(f"{'method':10s} {'params':20s} {'rule':5s}" + ''.join(f'{c:>16s}' for c in cols))
8  print(f"{'prepared':10s} {'(no filter)':20s} {'-':5s}" + ''.join(f'{base[c].mean():8.2f} ± {base[c
9  ↪ }.std():5.2f}' for c in cols[:-1]) + f'{"-":>16s}')
10 for n in rank:
11     params, ok = tuned[n]
12     print(f"{n:10s} {str(params):20s} {'ok' if ok else 'NO':5s}" +
13           ''.join(f'{results[n][c].mean():8.2f} ± {results[n][c].std():5.2f}' for c in cols))
14 passing = [n for n in rank if tuned[n][1]]
15 top3 = passing[:3]
16 print('\nmethods that fail the edge rule:', [n for n in rank if not tuned[n][1]] or 'none')
17 print('top 3 by test gCNR (edge rule passed):', top3)

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 24

```

1  BLUE, INK, MUTED = '#2a78d6', '#0b0b0b', '#52514e'
2  fig, axes = plt.subplots(1, 3, figsize=(17, 5), sharey=True) # best gCNR at the top, same ord
3  y = np.arange(len(rank))[:-1]
4  ↪ er in all three
5  for ax, m, title, ref, ref_label in [
6      (axes[0], 'gCNR', 'gCNR (higher = better)', base['gCNR'].mean(), 'prepared'),
7      (axes[1], 'SI', 'speckle index (lower = better)', base['SI'].mean(), 'prepared'),
8      (axes[2], 'EdgeRatio', 'edge sharpness ratio (rule ≥ 0.90)', EDGE_MIN, 'rule 0.90')]:

```

```

9     mean = [results[n][m].mean() for n in rank]; std = [results[n][m].std() for n in rank]
10    ax.barh(y, mean, height=0.6, color=BLUE, xerr=std, error_kw=dict(ecolor=MUTED, lw=1, capsize=2
11    ↪ ))
12    ax.axvline(ref, color=INK, ls='--', lw=1); ax.text(ref, len(rank) - 0.4, f' {ref_label}', colo
13    ↪ r=MUTED, fontsize=9)
14    for yi, v in zip(y, mean): ax.text(v, yi + 0.33, f'{v:.2f}', fontsize=8, color=MUTED, ha='cent
15    ↪ er')
16    ax.set_title(title, fontsize=11); ax.grid(axis='x', color='#e5e4e0', lw=0.8); ax.set_axisbelow
17    ↪ (True)
18    for sp in ('top', 'right'): ax.spines[sp].set_visible(False)
19    axes[0].set_yticks(y); axes[0].set_yticklabels([n + ('' if tuned[n][1] else ' x') for n in rank])
20    fig.suptitle('Breast test set (10 images), tuned params, mean ± std (x = fails edge rule on tune
21    ↪ set)')
22    plt.tight_layout(); plt.show()

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 25

```

1  show = test_set[:3]
2  fig, axes = plt.subplots(len(show), 1 + len(top3), figsize=(5 * (1 + len(top3)), 4.3 * len(show)))
3  for row, s in zip(axes, show):
4      ims = [(('before (prepared)', s['clean'], s['base'])) + \
5              [(f'{n} {tuned[n][0]}', out, metrics(out, s)) for n in top3 for out in [FILTERS[n](s['cl
6      ↪ ean'], s['scan'], **tuned[n][0])]]
7      for ax, (t, im, r) in zip(row, ims):
8          ax.imshow(im, cmap='gray', vmin=0, vmax=255); ax.contour(s['mask'], [0.5], colors='r', lin
9      ↪ ewidths=0.6)
10         ax.set_title(t, fontsize=11); ax.set_xticks([]); ax.set_yticks([])
11         ax.set_xlabel(f'gCNR {r['gCNR']:.2f} SI {r['SI']:.2f} Edge {r['Edge']} / s['base']['Edg
12     ↪ e']:.2f)x", fontsize=10)
13         row[0].set_ylabel(s['name'], fontsize=10)
14     fig.suptitle('Top 3 methods on 3 test images (red = lesion)'); plt.tight_layout(); plt.show()

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 27

```

1  def show_table(rows, title):
2      print(title); print(f"{'method':22s} {'params':18s} {'rule':5s}" + ''.join(f'{c:>16s}' for c i
3      ↪ n cols))
4      for label, (params, ok, res) in rows.items():
5          print(f"{label:22s} {params:18s} {ok:5s}" +
6                ''.join(f'{res[c].mean():8.2f} ± {res[c].std():5.2f}' if c in res else f'{"-":>16s}'
7      ↪ for c in cols))
8
9  fmt = lambda p: ', '.join(f'{k} {v:g}' for k, v in p.items())
10  wav = tuned['Wavelet'][0]
11  FILTERS['Hybrid-seq'] = lambda img, scan, eps=0.001: guided(wavelet(img, scan, **wav), scan, eps=e
12  ↪ ps)
13  FILTERS['Hybrid-joint'] = lambda img, scan, eps=0.001: guided(img, scan, eps=eps, guide=wavelet(im
14  ↪ g, scan, **wav))
15  HYB_GRID = {'eps': [0.0005, 0.001, 0.002, 0.005]}
16  tune_log = {}
17
18  tune_cnr = {'Guided': next(c for v, c, e in tune('Guided', GRIDS['Guided'], tune_set)[2] if v == t
19  ↪ uned['Guided'][0]['eps'])}
20  for n in ['Hybrid-seq', 'Hybrid-joint']:
21      params, ok, log = tune(n, HYB_GRID, tune_set)
22      print(f"{n:12s} " + ' '.join(f"{v}: CNR {c:.3f} edge {e:.2f}" for v, c, e in log) + f" -> {p
23  ↪ arams} rule {'ok' if ok else 'FAILED'}")
24      tuned[n] = (params, ok); tune_log[n] = log; results[n] = evaluate(n, params, test_set)
25      if ok: tune_cnr[n] = next(c for v, c, e in log if v == params['eps'])
26  BEST = max(tune_cnr, key=tune_cnr.get)
27  print(f"\ntune-set CNR: {'', '.join(f'{k} {v:.3f}' for k, v in tune_cnr.items())} -> BEST = {BEST
28  ↪ }\n")
29  show_table({'prepared (no filter)': ('-', '-', base)} |
30             {n: (fmt(tuned[n][0]), 'ok' if tuned[n][1] else 'NO', results[n]) for n in ['Guided', '
31  ↪ Hybrid-seq', 'Hybrid-joint']},
32             'Hybrid vs single best, breast test set (10 images), mean ± std')

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 28

```

1  def enhance(img, mask, clip=2.0, amount=1.0, sigma=2.0):
2      out = (cv2.createCLAHE(clipLimit=clip, tileGridSize=(8, 8)).apply(img) if clip > 0 else img).a
3      ↪ stype(np.float32) # clip 0 = no CLAHE
4      base = guided(out.astype(np.uint8), mask, r=4, eps=0.01).astype(np.float32) # edges without

```

```

5     ↳ speckle
6     return finish(out + amount * (base - cv2.GaussianBlur(base, (0, 0), sigma)), img, mask)
7
8 FILTERS['Enhance only'] = lambda img, scan: enhance(img, scan)
9 FILTERS[f'{BEST} + enhance'] = lambda img, scan, **p: enhance(FILTERS[BEST](img, scan, **p), scan)
10 for n, p in [(('Enhance only', {}), (f'{BEST} + enhance', tuned[BEST][0]))]:
11     results[n] = evaluate(n, p, test_set)
12 show_table({'prepared (no filter)': ('-', '-', base),
13            'Enhance only': ('-', '-', results['Enhance only']),
14            BEST: (fmt(tuned[BEST][0]), '', results[BEST]),
15            f'{BEST} + enhance': (fmt(tuned[BEST][0]), '', results[f'{BEST} + enhance'])},
16            'Without / with enhance(), breast test set (10 images), mean ± std')
17
18 for clip in [0, 1.0, 2.0]:
19     ↳ sharpening?
20     FILTERS[f'{BEST} + enh clip {clip:g}'] = lambda img, scan, c=clip: enhance(FILTERS[BEST](img,
21     ↳ scan, **tuned[BEST][0]), scan, clip=c)
22     results[f'{BEST} + enh clip {clip:g}'] = evaluate(f'{BEST} + enh clip {clip:g}', {}, test_set)
23 print()
24 show_table({BEST: (fmt(tuned[BEST][0]), '', results[BEST])} |
25            {f'{BEST} + enh clip {c:g}': ('sharpen only' if c == 0 else f'CLAHE {c:g}', '', results
26     ↳ [f'{BEST} + enh clip {c:g}']) for c in [0, 1.0, 2.0]}),
27            'enhance() variants on top of the best filter: CLAHE clip 0 (off) / 1 / 2, breast test
28     ↳ set')

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 29

```

1 def estimate_noise(img, scan_mask, k=7, w=31):
2     f = img.astype(np.float32)
3     mu = cv2.blur(f, (k, k)); cov = np.sqrt(np.maximum(cv2.blur(f * f, (k, k)) - mu * mu, 0)) / np
4     ↳ .maximum(mu, 1)
5     sm = cv2.blur(f, (9, 9)); mw = cv2.blur(sm, (w, w))
6     structure = np.sqrt(np.maximum(cv2.blur(sm * sm, (w, w)) - mw * mw, 0)) / np.maximum(mw, 1)
7     valid = (cv2.erode(scan_mask.astype(np.uint8), kernel(w)) > 0) & (mw > np.percentile(f[scan_ma
8     ↳ sk], 25))
9     homog = valid & (structure <= np.percentile(structure[valid], 20))
10    return float(np.median(cov[homog]))
11
12 EPS_FINE = [0.0002, 0.0005, 0.001, 0.002, 0.005, 0.01]
13 pts = []
14     ↳ ne image
15 for s in tune_set:
16     ok = [(e, r['CNR']) for e in EPS_FINE for r in [metrics(FILTERS[BEST](s['clean'], s['scan'], e
17     ↳ ps=e), s)]
18     if r['Edge'] >= EDGE_MIN * s['base']['Edge']
19     pts.append((estimate_noise(s['clean'], s['scan']), max(ok, key=lambda t: t[1])[0] if ok else E
20     ↳ PS_FINE[0]))
21 ln, le = np.log(np.array(pts)).T
22 b, a = np.polyfit(ln, le, 1)
23
24 def auto_params(noise_level):
25     return {'eps': float(np.clip(np.exp(a + b * np.log(noise_level)), EPS_FINE[0], EPS_FINE[-1]))}
26
27 print(f'fit on tune set: log(eps) = {a:.2f} + {b:.2f}·log(noise), r = {np.corrcoef(ln, le)[0, 1]:
28     ↳ .2f}')
29 print('tune images (noise, best eps):', ', '.join(f'({n:.3f}, {e:g})' for n, e in pts))
30 FILTERS[f'{BEST} auto'] = lambda img, scan: FILTERS[BEST](img, scan, **auto_params(estimate_noise(
31     ↳ img, scan)))
32 results[f'{BEST} auto'] = evaluate(f'{BEST} auto', {}, test_set)
33 test_eps = [auto_params(estimate_noise(s['clean'], s['scan']))['eps'] for s in test_set]
34 print(f'auto eps on test images: {"", ".join(f"{e:.4f}" for e in test_eps)}\n')
35 show_table({'prepared (no filter)': ('-', '-', base),
36            f'{BEST} fixed': (fmt(tuned[BEST][0]), '', results[BEST]),
37            f'{BEST} auto': ('eps from noise', '', results[f'{BEST} auto'])},
38            'Fixed vs auto params, breast test set (10 images), mean ± std (auto ms includes estima
39     ↳ te_noise)')

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 30

```

1 BLUE, INK, MUTED = '#2a78d6', '#0b0b0b', '#52514e'
2 fig, ax = plt.subplots(figsize=(6, 4))
3 ax.scatter(*np.array(pts).T, s=40, color=BLUE, label='tune images')
4 xs = np.linspace(ln.min(), ln.max(), 50); ax.plot(np.exp(xs), np.exp(a + b * xs), color=INK, lw=2,
5     ↳ label='fit')

```

```

6 ax.set_xscale('log'); ax.set_yscale('log'); ax.set_xlabel('estimated noise (median local CoV)'); a
7   ↳ x.set_ylabel('best eps')
8 ax.set_title(f'auto_params fit for {BEST} (tune set)', fontsize=11); ax.legend(frameon=False)
9 for sp in ('top', 'right'): ax.spines[sp].set_visible(False)
10 plt.tight_layout(); plt.show()
11
12 show = test_set[:2]
13 fig, axes = plt.subplots(len(show), 5, figsize=(25, 5.6 * len(show)))
14 for row, s in zip(axes, show):
15     ims = [('before (prepared)', s['clean']),
16            (f'{BEST} fixed ({fmt(tuned[BEST][0])})', FILTERS[BEST](s['clean'], s['scan'], **tuned[
17   ↳ BEST][0])),
18            (f'{BEST} auto (eps {auto_params(estimate_noise(s["clean"], s["scan"]))["eps"]:.4f})',
19   ↳ FILTERS[f'{BEST} auto'](s['clean'], s['scan'])),
20            (f'{BEST} + sharpen only', FILTERS[f'{BEST} + enh clip 0'](s['clean'], s['scan'])),
21            (f'{BEST} + enhance (CLAHE 2)', FILTERS[f'{BEST} + enhance'](s['clean'], s['scan'], **t
22   ↳ uned[BEST][0]))]
23     for ax, (t, im) in zip(row, ims):
24         r = metrics(im, s)
25         ax.imshow(im, cmap='gray', vmin=0, vmax=255); ax.contour(s['mask'], [0.5], colors='r', lin
26   ↳ ewidths=0.6)
27         ax.set_title(t, fontsize=11); ax.set_xticks([]); ax.set_yticks([])
28         ax.set_xlabel(f'gCNR {r["gCNR"]:.2f} SI {r["SI"]:.2f} Edge {r["Edge"]} / s["base"]['Edg
29   ↳ e']:.2f}x", fontsize=10)
30         row[0].set_ylabel(s['name'], fontsize=10)
31 fig.suptitle('Step 6 on 2 test images (red = lesion)'); plt.tight_layout(); plt.show()

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 32

```

1 ENH_CLIP = 0
2 HYBRID = max(['Hybrid-seq', 'Hybrid-joint'], key=lambda n: next(e for v, c, e in tune_log[n] if v
3   ↳ == tuned[n][0]['eps']))
4 FILTERS['Final (auto)'] = lambda img, scan: FILTERS[f'{BEST} auto'](img, scan)
5 FILTERS['Final + enhance'] = lambda img, scan: enhance(FILTERS['Final (auto)'](img, scan), scan, c
6   ↳ lip=ENH_CLIP)
7 FINAL_METHODS = {'prepared (no filter)': None, **{n: tuned[n][0] for n in top3}, HYBRID: tuned[HYB
8   ↳ RID][0],
9   ↳ 'Final (auto)': {}, 'Final + enhance': {}}
10
11 def run_organ(name, params, data):
12     res = []
13     for s in data:
14         if params is None: r = dict(s['base'], ms=np.nan)
15         else:
16             out, ms = timed(FILTERS[name], s['clean'], s['scan'], **params); r = metrics(out, s, m
17   ↳ s)
18             if 'Edge' in r: r['EdgeRatio'] = r['Edge'] / s['base']['Edge']
19             res.append(r)
20     return {m: np.array([r.get(m, np.nan) for r in res]) for m in ['gCNR', 'SI', 'EdgeRatio', 'ms'
21   ↳ '']}
22
23 organ_data = {'breast': test_set, 'thyroid': datasets['thyroid'], 'kidney': datasets['kidney'], 'l
24   ↳ iver': datasets['liver']}
25 final = {(o, n): run_organ(n, p, d) for o, d in organ_data.items() for n, p in FINAL_METHODS.items
26   ↳ ()}
27
28 cell_fmt = lambda v: f'{np.nanmean(v):6.2f} ± {np.nanstd(v):4.2f}' if np.isfinite(v).any() else 'n
29   ↳ /a'
30 print(f"hybrid shown: {HYBRID}. mean ± std; breast = 10 test images, other organs = 20 images, no
31   ↳ re-tuning")
32 print(f"{'organ':8s} {'method':22s} + ''.join(f'{c:>18s}' for c in ['gCNR', 'SI', 'EdgeRatio', 'm
33   ↳ s'])")
34 for (o, n), r in final.items():
35     print(f"{'o':8s} {'n':22s} + ''.join(f'{cell_fmt(r[c]):>18s}' for c in ['gCNR', 'SI', 'EdgeRatio'
36   ↳ , 'ms'])")
37     if n == 'Final + enhance': print()
38
39 print(f'auto_params was fitted on breast tune noise {min(n for n, _ in pts):.3f}-{max(n for n, _ i
40   ↳ n pts):.3f}; eps is clipped to {EPS_FINE[0]}-{EPS_FINE[-1]}')
41 for o, d in organ_data.items():
42     nz = [estimate_noise(s['clean'], s['scan']) for s in d]; ep = [auto_params(n)['eps'] for n in
43   ↳ nz]
44     print(f"{'o':8s} noise {np.mean(nz):.3f} ({min(nz):.3f}-{max(nz):.3f}) auto eps {np.mean(ep):.
45   ↳ 4f} ({min(ep):.4f}-{max(ep):.4f})")

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 33

```

1 def post_process(img, method='auto', enhance=False):
2     clean, scan = prepare(to_gray(img))
3     params = auto_params(estimate_noise(clean, scan)) if method == 'auto' else tuned[method][0]
4     out = FILTERS[BEST if method == 'auto' else method](clean, scan, **params)
5     return sharpen(out, scan) if enhance else out
6
7 sharpen = lambda img, scan: enhance(img, scan, clip=ENH_CLIP)    # module-level enhance() (the arg
8     ↳ ument shadows it)
9 for o in ORGANS:                                                # check: same result as the pipe
10     ↳ line above, and timing
11     s = organ_data[o][0]
12     out, ms = timed(post_process, s['image'])
13     same = np.array_equal(out, FILTERS['Final (auto)'](s['clean'], s['scan']))
14     print(f'{o:8s} {s["name"]:32s} post_process: {out.dtype} {out.shape} {ms:6.1f} ms identical
15     ↳ to pipeline: {same}')

```

262SP009_Post_Image_Processing_v2.ipynb, code cell 34

```

1 from matplotlib.patches import Rectangle
2
3 def zoom_box(s, size=130, pad=30):                                # lesion box + pad (kidney: ENL R
4     ↳ 0I), at least size px
5     m = s['mask'] if s['mask'] is not None else s['roi']
6     ys, xs = np.nonzero(m); cy, cx = (ys.min() + ys.max()) // 2, (xs.min() + xs.max()) // 2
7     h, w = max(np.ptp(ys) + 2 * pad, size) // 2, max(np.ptp(xs) + 2 * pad, size) // 2
8     return max(cy - h, 0), min(cy + h, m.shape[0]), max(cx - w, 0), min(cx + w, m.shape[1])
9
10 fig, axes = plt.subplots(8, 3, figsize=(16, 42))
11 for k, o in enumerate(ORGANS):
12     s = next((x for x in organ_data[o] if x['mask'] is not None), organ_data[o][0])
13     ims = [('original (metrics: prepared input)', s['image']), ('best: post_process()', post_proce
14     ↳ ss(s['image']))],
15           ('best + enhance', post_process(s['image'], enhance=True))]
16     y0, y1, x0, x1 = zoom_box(s)
17     for ax, zax, (t, im) in zip(axes[2 * k], axes[2 * k + 1], ims):
18         r = s['base'] if t.startswith('original') else metrics(im, s)
19         lab = f"SI {r['SI']:.2f}"
20         if s['mask'] is not None: lab = f"gCNR {r['gCNR']:.2f} " + lab + f" Edge {r['Edge'] /
21     ↳ s['base']['Edge']:.2f}x"
22         ax.imshow(im, cmap='gray', vmin=0, vmax=255); ax.set_title(t, fontsize=12); ax.set_xlabel(
23     ↳ lab, fontsize=10)
24         ax.add_patch(Rectangle((x0, y0), x1 - x0, y1 - y0, fill=False, ec='yellow', lw=1.5))
25         zax.imshow(im[y0:y1, x0:x1], cmap='gray', vmin=0, vmax=255, interpolation='nearest')
26         zax.set_title(f'zoom: {t.split(" ")[0]}', fontsize=11)
27         for a_, m_ in [(ax, s['mask']), (zax, None if s['mask'] is None else s['mask'][y0:y1, x0:x
28     ↳ 1])]:
29             if m_ is not None: a_.contour(m_, [0.5], colors='r', linewidths=0.6)
30             a_.set_xticks([]); a_.set_yticks([])
31             axes[2 * k, 0].set_ylabel(f'{o}\n{s["name"].split("/")[-1]}', fontsize=11)
32             axes[2 * k + 1, 0].set_ylabel(f'{o} zoom\n(' + ('lesion' if s['mask'] is not None else 'flat t
33     ↳ issue') + ')', fontsize=11)
34 fig.suptitle('Final pipeline, one image per organ: full image + zoom of the yellow box (red = lesi
35     ↳ on / nodule; kidney has no mask)', y=1.0)
36 plt.tight_layout(); plt.show()

```

run_team8.py (runner written for this demo)

```

1 """
2 run_team8.py - Team 8 demo: Post-Image Processing (despeckling).
3
4 One command runs the stage and saves every result into the output/ folder:
5
6     python3 run_team8.py      (macOS / Linux)
7     python run_team8.py      (Windows)
8
9 It loads two finished grayscale B-mode images from input/ (the output of the B-mode Image
10 Formation stage), despeckles them with postprocess() from code/post_processing.py (guided
11 filter, r = 4, eps = 0.001), and measures image quality before and after with the module's
12 own metric functions.
13
14 The team's files in code/ are NOT changed; this script only calls them.
15 """

```

```

16 import io
17 import json
18 import sys
19 import time
20 import types
21 from pathlib import Path
22
23 import matplotlib
24
25 matplotlib.use("Agg") # save figures to files, do not open windows
26 import matplotlib.pyplot as plt # noqa: E402
27 import numpy as np # noqa: E402
28
29 HERE = Path(__file__).resolve().parent
30 CODE, INPUT, OUTPUT = HERE / "code", HERE / "input", HERE / "output"
31 OUTPUT.mkdir(exist_ok=True)
32
33
34 # -----
35 # Console helper: everything printed goes to the screen AND to output/console_log.txt
36 # -----
37 class Tee(io.TextIOBase):
38     def __init__(self, *streams):
39         self.streams = streams
40
41     def write(self, s):
42         for st in self.streams:
43             st.write(s)
44         return len(s)
45
46     def flush(self):
47         for st in self.streams:
48             st.flush()
49
50
51 LOG = io.StringIO()
52 REAL_STDOUT = sys.stdout
53 sys.stdout = Tee(REAL_STDOUT, LOG)
54 SAVED = [] # list of (file name, meaning)
55
56
57 def save(fig, name, meaning):
58     fig.savefig(OUTPUT / name, dpi=130, bbox_inches="tight")
59     plt.close(fig)
60     SAVED.append((name, meaning))
61     print(f" saved output/{name}")
62
63
64 def banner(text):
65     print("\n" + "=" * 78 + f"\n{text}\n" + "=" * 78)
66
67
68
69 t_start = time.time()
70 banner("TEAM 8: POST-IMAGE PROCESSING (NITK-UsoundSim)")
71 print(f"Python {sys.version.split()[0]}, numpy {np.__version__}, matplotlib {matplotlib.__version__}")
72 print(f"Folder: {HERE.name}")
73
74 # =====
75 # Post-Image Processing (despeckling with the guided filter)
76 # =====
77 banner("Post-Image Processing (code/post_processing.py)")
78 sys.dont_write_bytecode = True # keep code/ clean (no __pycache__ folder)
79 sys.path.insert(0, str(CODE))
80 try:
81     import config # noqa: F401 (the full project's settings file)
82 except ImportError:
83     # post_processing.py does "import config" for its scan_convert() function, which belongs to
84     # the B-mode team and is NOT used in this demo. The project's config.py is not part of this
85     # folder, so an empty placeholder module is registered to let the file import unchanged.
86     sys.modules["config"] = types.ModuleType("config")
87     print("Note: project config.py not in this folder -> empty placeholder used")
88     print(" (only scan_convert() needs it, and scan_convert() is not used in this demo).")
89 import post_processing as pp # noqa: E402
90
91 print("Filter: postprocess(image) with its defaults -> method='guided', r=4, eps=0.001, gamma=1.0\

```

```

    ↪ n")
92
93 data = {}
94 for name in ["cyst_lesion", "scatterers"]:
95     z = np.load(INPUT / f"{name}_image.npz")
96     img = z["image"].astype(float)
97     t0 = time.time()
98     out = pp.postprocess(img)
99     ms = 1000 * (time.time() - t0)
100     data[name] = dict(img=img, out=out, x=z["x_img"] * 1e3, z=z["z_img"] * 1e3, ms=ms)
101     print(f"{name:12s}: input {img.shape}, range {img.min():.3f}-{img.max():.3f} -> output range
    ↪ "
102         f"{out.min():.3f}-{out.max():.3f}, time {ms:.1f} ms")
103
104 # Lesion mask for the cyst: circle, radius 6 mm, centre (x = 0, z = 30) mm on the image grid
105 C = data["cyst_lesion"]
106 XX, ZZ = np.meshgrid(C["x"], C["z"])
107 lesion = (XX - 0.0) ** 2 + (ZZ - 30.0) ** 2 <= 6.0 ** 2
108 ring = pp.lesion_ring(lesion)
109 print(f"\nCyst lesion mask: circle r = 6 mm at (0, 30) mm -> {lesion.sum()} px; background ring "
110       f"{pp.RING} px wide -> {ring.sum()} px")
111
112
113 def measure(img01):
114     u8 = pp.to_uint8(img01)
115     f = u8.astype(float) / 255
116     return dict(SI=pp.speckle_index(u8), contrast_db=50 * (f[lesion].mean() - f[ring].mean()),
117               CNR=pp.cnr(u8, lesion), gCNR=pp.gcnr(u8, lesion), edge=pp.edge_sharpness(u8, lesio
    ↪ n))
118
119
120 print("\nQuality metrics (computed on uint8 images with the module's own functions):")
121 print(f"{'image':12s} {'metric':34s} {'before':>9s} {'after':>9s} {'change':>10s}")
122 print("-" * 78)
123 for name in ["scatterers", "cyst_lesion"]:
124     D = data[name]
125     si0, si1 = pp.speckle_index(pp.to_uint8(D["img"])), pp.speckle_index(pp.to_uint8(D["out"]))
126     D["SI"] = (si0, si1)
127     print(f"{name:12s} {'speckle index SI (lower = smoother)':34s} {si0:9.3f} {si1:9.3f} {100 * (s
    ↪ i1 / si0 - 1):+9.1f}%")
128 m0, m1 = measure(C["img"]), measure(C["out"])
129 C["m0"], C["m1"] = m0, m1
130 for key, label in [("contrast_db", "contrast lesion-ring (dB, 50 dB DR)"), ("CNR", "CNR (higher =
    ↪ better)"),
131                   ("gCNR", "gCNR (0..1, higher = better)"), ("edge", "edge sharpness (Sobel, grey
    ↪ /px)"]):
132     ch = m1[key] - m0[key]
133     print(f"{'cyst_lesion':12s} {label:34s} {m0[key]:9.3f} {m1[key]:9.3f} {ch:+10.3f}")
134 print(f"{'cyst_lesion':12s} {'edge sharpness kept':34s} {'':9s} {100 * m1['edge'] / m0['edge']:8.1
    ↪ f}%")
135
136 # eps comparison on the cyst
137 print("\nEffect of eps (r = 4) on the cyst image:")
138 print(f"{'eps':>8s} {'SI':>8s} {'gCNR':>8s} {'CNR':>8s} {'edge kept':>10s}")
139 eps_list = [0.001, 0.002, 0.005, 0.01]
140 eps_res = {}
141 for eps in eps_list:
142     o = pp.postprocess(C["img"], eps=eps)
143     m = measure(o)
144     eps_res[eps] = (o, m)
145     print(f"{'eps':8.3f} {m['SI']:8.3f} {m['gCNR']:8.3f} {m['CNR']:8.3f} {100 * m['edge'] / m0['edge
    ↪ ']:9.1f}%")
146
147 # ----- figures
148 print("\nSaving figures:")
149 circle = lambda ax: ax.add_patch(plt.Circle((0, 30), 6, fill=False, ec="red", lw=0.8, ls="--"))
150
151 fig, ax = plt.subplots(1, 4, figsize=(15, 7.5))
152 for k, name in enumerate(["scatterers", "cyst_lesion"]):
153     D = data[name]
154     ext = [D["x"][0], D["x"][-1], D["z"][-1], D["z"][0]]
155     for j, (im, t) in enumerate([(D["img"], "before"), (D["out"], "after guided filter")]):
156         a = ax[2 * k + j]
157         a.imshow(im, cmap="gray", vmin=0, vmax=1, extent=ext)
158         si = D["SI"][j]
159         a.set_title(f"{name}\n{t}\nSI = {si:.3f}", fontsize=10); a.set_xlabel("x (mm)")
160         if name == "cyst_lesion":

```

```

161         circle(a)
162 ax[0].set_ylabel("depth z (mm)")
163 fig.suptitle("Before / after despeckling (postprocess, r = 4, eps = 0.001); red dashed = cyst mask
↳ ", fontsize=11)
164 save(fig, "B1_before_after.png", "Both input images before and after the guided filter, with speck
↳ le index.")
165
166 y0, y1 = np.searchsorted(C["z"], [22, 38])
167 x0, x1 = np.searchsorted(C["x"], [-8, 8])
168 fig, ax = plt.subplots(1, 2, figsize=(11, 5.8))
169 for a, im, t in [(ax[0], C["img"], "before"), (ax[1], C["out"], "after")]:
170     a.imshow(im[y0:y1, x0:x1], cmap="gray", vmin=0, vmax=1, interpolation="nearest",
171             extent=[C["x"][x0], C["x"][x1 - 1], C["z"][y1 - 1], C["z"][y0]])
172     circle(a); a.set_xlabel("x (mm)"); a.set_ylabel("z (mm)")
173     a.set_title(f"cyst zoom {t}: gCNR {C['m0'] if t == 'before' else 'm1'}['gCNR']:.3f}, "
174               f"edge {C['m0'] if t == 'before' else 'm1'}['edge']:.1f}", fontsize=10)
175 fig.suptitle("Zoom on the cyst edge (x -8..8 mm, z 22..38 mm)", fontsize=11)
176 save(fig, "B2_cyst_edge_zoom.png", "Close-up of the cyst border before and after: speckle smoother
↳ , border kept.")
177
178 fig, ax = plt.subplots(1, 2, figsize=(11, 6.5))
179 for a, name in zip(ax, ["scatterers", "cyst_lesion"]):
180     D = data[name]
181     diff = D["out"] - D["img"]
182     lim = np.percentile(np.abs(diff), 99)
183     im = a.imshow(diff, cmap="seismic", vmin=-lim, vmax=lim, extent=[D["x"][0], D["x"][-1], D["z"]
↳ [-1], D["z"][0]])
184     a.set_title(f"{name}: after - before\n(mean |diff| = {np.abs(diff).mean():.3f})", fontsize=10)
185     a.set_xlabel("x (mm)"); plt.colorbar(im, ax=a, fraction=0.046, label="brightness change")
186 ax[0].set_ylabel("z (mm)")
187 fig.suptitle("Difference map (after - before): red = pixel became brighter, blue = darker", fontsi
↳ ze=11)
188 save(fig, "B3_difference_map.png", "After minus before: the filter removes fine speckle grains eve
↳ rywhere.")
189
190 row = int(np.argmin(np.abs(C["z"] - 30)))
191 fig, ax = plt.subplots(figsize=(11, 4.2))
192 ax.plot(C["x"], C["img"][row], color="0.6", lw=1, label="before")
193 ax.plot(C["x"], C["out"][row], color="tab:red", lw=1.6, label="after guided filter")
194 for xe in (-6, 6):
195     ax.axvline(xe, color="k", ls="--", lw=0.8)
196 ax.set_xlabel("x (mm)"); ax.set_ylabel("brightness (0-1)"); ax.grid(alpha=0.3); ax.legend()
197 ax.set_title(f"Brightness profile along row z = {C['z'][row]:.2f} mm through the cyst centre (dash
↳ ed = cyst edges)")
198 save(fig, "B4_brightness_profile.png", "One image row through the cyst: fluctuations reduced, edge
↳ step kept.")
199
200 fig, ax = plt.subplots(1, 2, figsize=(12, 4.2))
201 bins = np.linspace(0, 1, 65)
202 for a, name in zip(ax, ["scatterers", "cyst_lesion"]):
203     D = data[name]
204     a.hist(D["img"].ravel(), bins, alpha=0.55, color="0.4", label="before")
205     a.hist(D["out"].ravel(), bins, alpha=0.55, color="tab:red", label="after")
206     a.set_title(name); a.set_xlabel("brightness (0-1)"); a.set_ylabel("pixel count"); a.legend()
207 fig.suptitle("Grey-level histograms before / after: fewer extreme values (tails and the spike at 0
↳ shrink)", fontsize=11)
208 save(fig, "B5_histograms.png", "Grey-level histograms before/after for both images.")
209
210 fig = plt.figure(figsize=(14, 7.5))
211 gs = fig.add_gridspec(2, 4)
212 for k, eps in enumerate(eps_list):
213     a = fig.add_subplot(gs[k // 2, k % 2])
214     a.imshow(eps_res[eps][0][y0:y1, x0:x1], cmap="gray", vmin=0, vmax=1,
215             extent=[C["x"][x0], C["x"][x1 - 1], C["z"][y1 - 1], C["z"][y0]])
216     m = eps_res[eps][1]
217     a.set_title(f"eps = {eps}: SI {m['SI']:.3f}, gCNR {m['gCNR']:.3f}", fontsize=9); a.set_xticks(
↳ []); a.set_yticks([])
218 a = fig.add_subplot(gs[:, 2:])
219 er = [100 * eps_res[e][1]["edge"] / m0["edge"] for e in eps_list]
220 gc = [eps_res[e][1]["gCNR"] for e in eps_list]
221 a.plot(eps_list, er, "o-", color="tab:blue", label="edge kept (%)")
222 a.set_xscale("log"); a.set_xlabel("eps"); a.set_ylabel("edge kept (%)", color="tab:blue"); a.grid(
↳ alpha=0.3)
223 a2 = a.twinx(); a2.plot(eps_list, gc, "s--", color="tab:red", label="gCNR"); a2.set_ylabel("gCNR",
↳ color="tab:red")
224 a.set_title("bigger eps = more smoothing, less edge")
225 fig.suptitle("eps comparison on the cyst (zoom x -8..8, z 22..38 mm)", fontsize=11)

```

```

226 fig.tight_layout()
227 save(fig, "B6_eps_comparison.png", "Cyst zoom for eps 0.001/0.002/0.005/0.01 and a plot of edge ke
↳ pt and gCNR vs eps.")
228
229 np.savez(OUTPUT / "B_postprocessed_images.npz", cyst_lesion=C["out"].astype(np.float32),
230         scatterers=data["scatterers"]["out"].astype(np.float32))
231 SAVED.append(("B_postprocessed_images.npz", "The two despeckled images as numbers (float32, 0-1)."
↳ ))
232 print(" saved output/B_postprocessed_images.npz")
233
234 # =====
235 banner("SUMMARY")
236 print(f"Cyst image: SI {C['SI'][0]:.3f} -> {C['SI'][1]:.3f}, gCNR {m0['gCNR']:.3f} -> {m1['gCNR']:.
↳ .3f}, "
237         f"edge kept {100 * m1['edge'] / m0['edge']:.0f} %")
238 print(f"Files written to output/: {len(SAVED) + 2} (including console_log.txt and console_screensh
↳ ot.png)")
239 print(f"Total run time: {time.time() - t_start:.1f} s")
240
241 # ----- console log + screenshot
242 # ----- console log + screenshot
243 sys.stdout = REAL_STDOUT
244 text = LOG.getvalue()
245 (OUTPUT / "console_log.txt").write_text(text, encoding="utf-8")
246
247
248 def console_png(text, out_png, width=110):
249     lines = []
250     for ln in text.rstrip("\n").splitlines():
251         while len(ln) > width:
252             lines.append(ln[:width]); ln = " " + ln[width:]
253         lines.append(ln)
254     h = 0.6 + 0.16 * len(lines)
255     fig = plt.figure(figsize=(0.083 * width + 0.4, h))
256     fig.patch.set_facecolor("#1e1e1e")
257     fig.text(0.01, 1 - 0.28 / h, "Terminal: python3 run_team8.py", color="#9a9a9a", fontsize=9, fa
↳ mily="monospace",
258             va="center")
259     for i, ln in enumerate(lines):
260         fig.text(0.01, 1 - (0.55 + 0.16 * i) / h, ln, color="#e6e6e6", fontsize=8.4, family="monos
↳ pace", va="top")
261     fig.savefig(out_png, dpi=130, facecolor=fig.get_facecolor())
262     plt.close(fig)
263
264
265 console_png(text, OUTPUT / "console_screenshot.png")
266 print("Saved output/console_log.txt and output/console_screenshot.png")

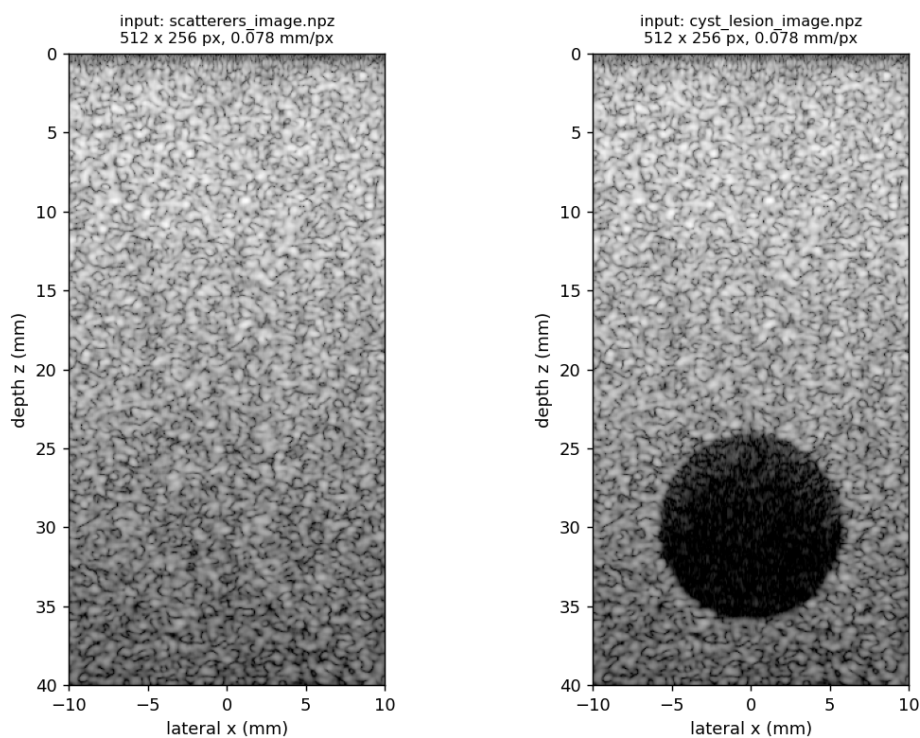
```

4 Input

The finished grayscale B-mode images

The input of this stage is the grayscale B-mode image made by the B-mode Image Formation stage. Both files come from the project's saved simulator results (results/cyst_lesion.npz and results/scatterers.npz, key 'image'): scan-converted, log-compressed B-mode images of 512 x 256 pixels, brightness 0-1, pixel size 0.078 mm, $x = -10..10$ mm, $z = 0..40$ mm. They were saved as float32 with their axes x_img and z_img (in metres).

- **scatterers**: uniform tissue made of random scatterers, i.e. pure speckle.
- **cyst_lesion**: the same tissue with an anechoic (echo-free) cyst of radius 6 mm centred at $x = 0$, $z = 30$ mm.



Input previews (input/scatterers_image_preview.png and input/cyst_lesion_image_preview.png).

5 Output

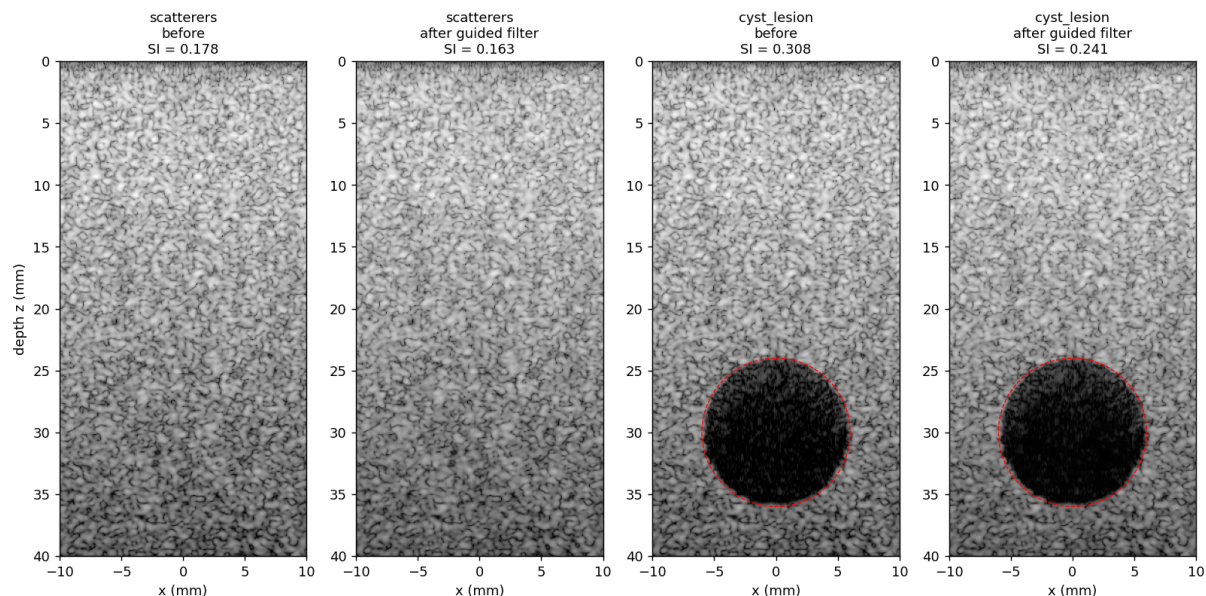
Everything below (except Section 5.3) was produced by run_team8.py

5.1 Quality metrics and figures

Image	Metric	Before	After	Change
scatterers	Speckle index SI (lower = smoother)	0.178	0.163	-8.3 %
cyst_lesion	Speckle index SI	0.308	0.241	-21.7 %
cyst_lesion	Contrast lesion - ring (dB)	-19.404	-19.317	+0.087
cyst_lesion	CNR	2.323	2.425	+0.102
cyst_lesion	gCNR	0.860	0.871	+0.010
cyst_lesion	Edge sharpness (Sobel, grey/px)	145.728	140.534	96.4 % kept

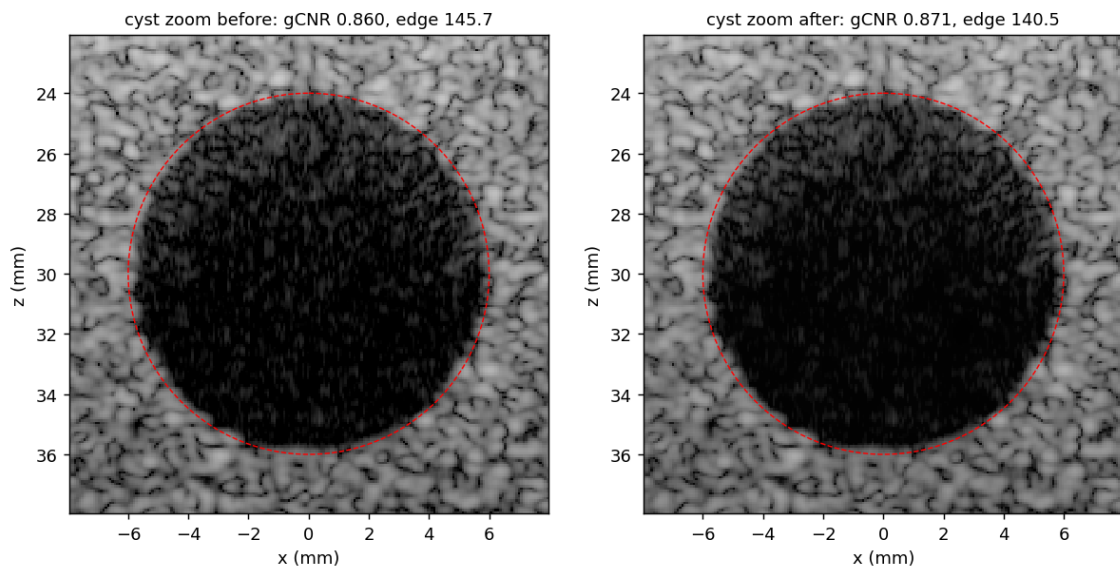
The numbers are copied from output/console_log.txt of the run used for this document. They match the values measured earlier in the project (SI 0.308 -> 0.241, CNR 2.323 -> 2.425, gCNR 0.860 -> 0.871, edge kept 96 %). The contrast in dB is almost unchanged (the filter keeps the mean levels), while the speckle spread goes down, so CNR and gCNR rise a little. The SI of the scatterers image drops less (8 %) than that of the cyst image (22 %).

Before / after despeckling (postprocess, $r = 4$, $\epsilon = 0.001$); red dashed = cyst mask

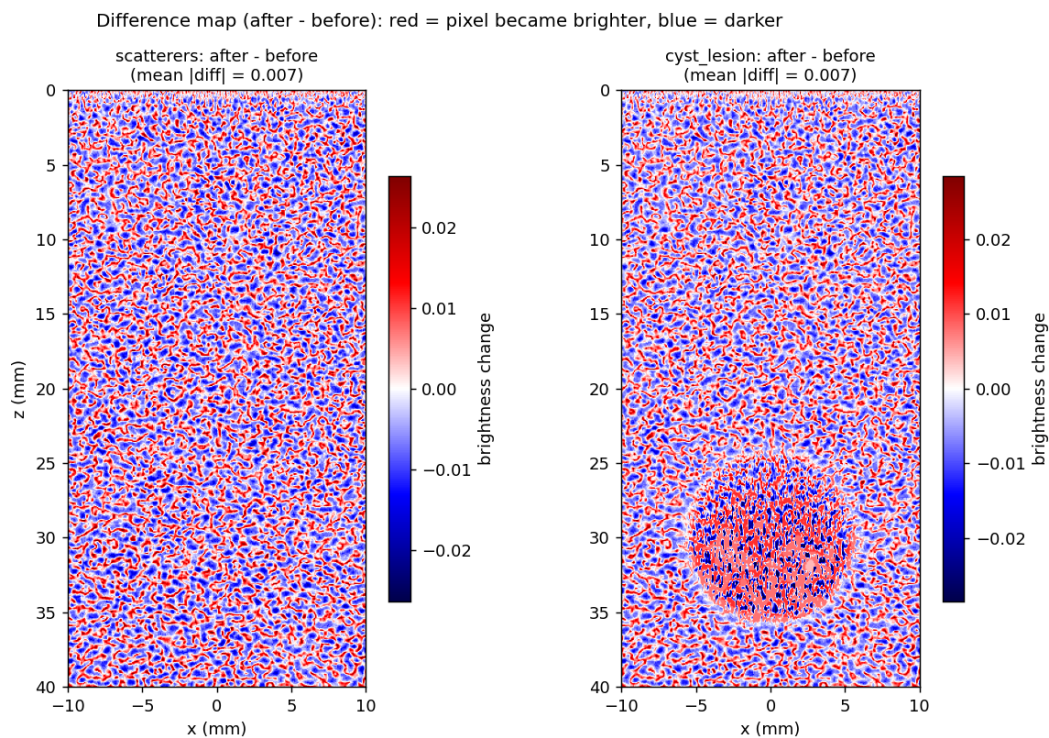


B1: both inputs before and after postprocess(). The grain becomes a little smoother; the cyst border (red dashed circle = mask used for metrics) stays sharp.

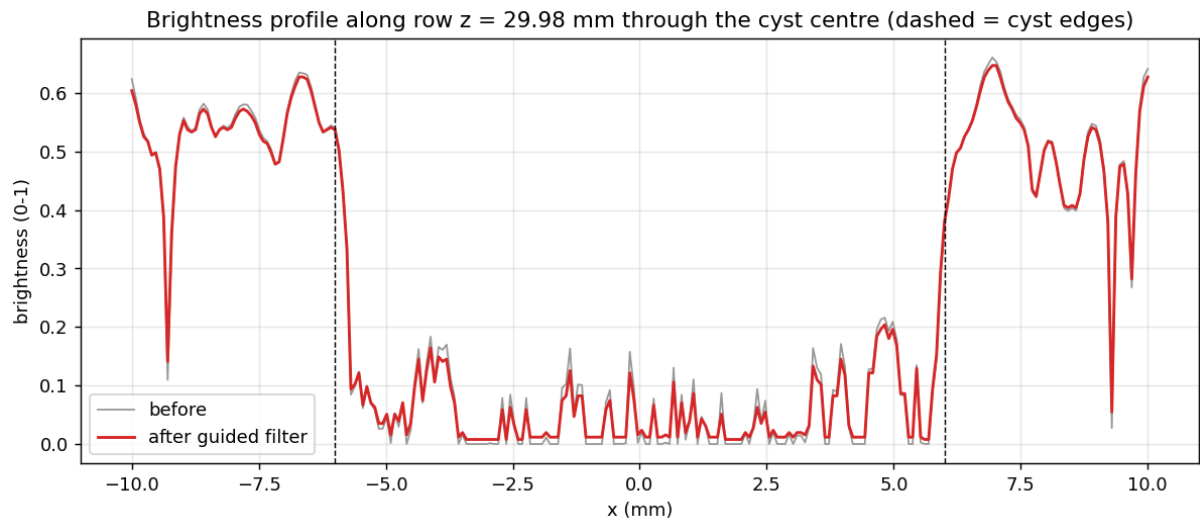
Zoom on the cyst edge (x -8..8 mm, z 22..38 mm)



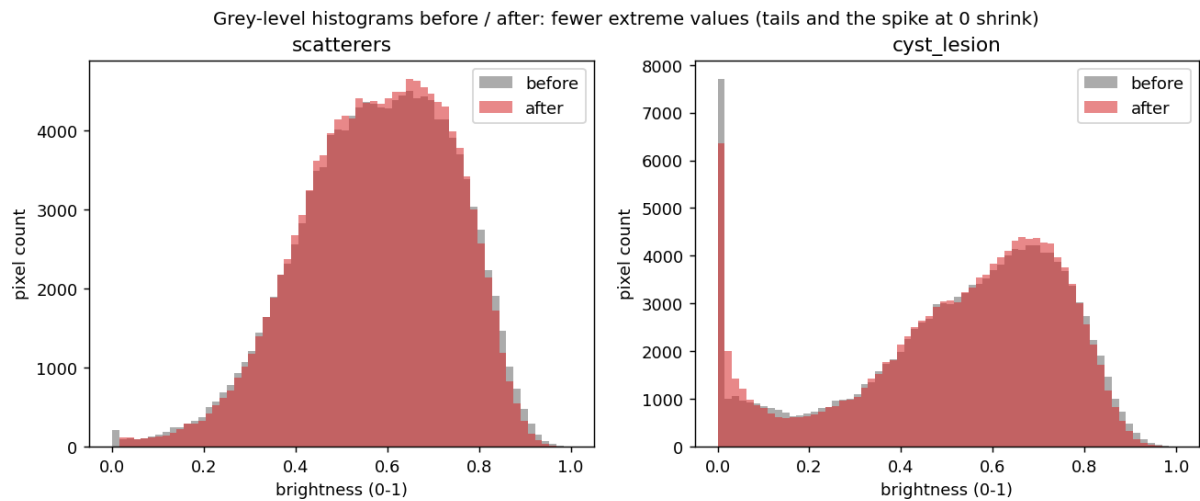
B2: zoom on the cyst (x -8..8 mm, z 22..38 mm). The border keeps its shape; small dark speckle holes in the tissue are partly filled.



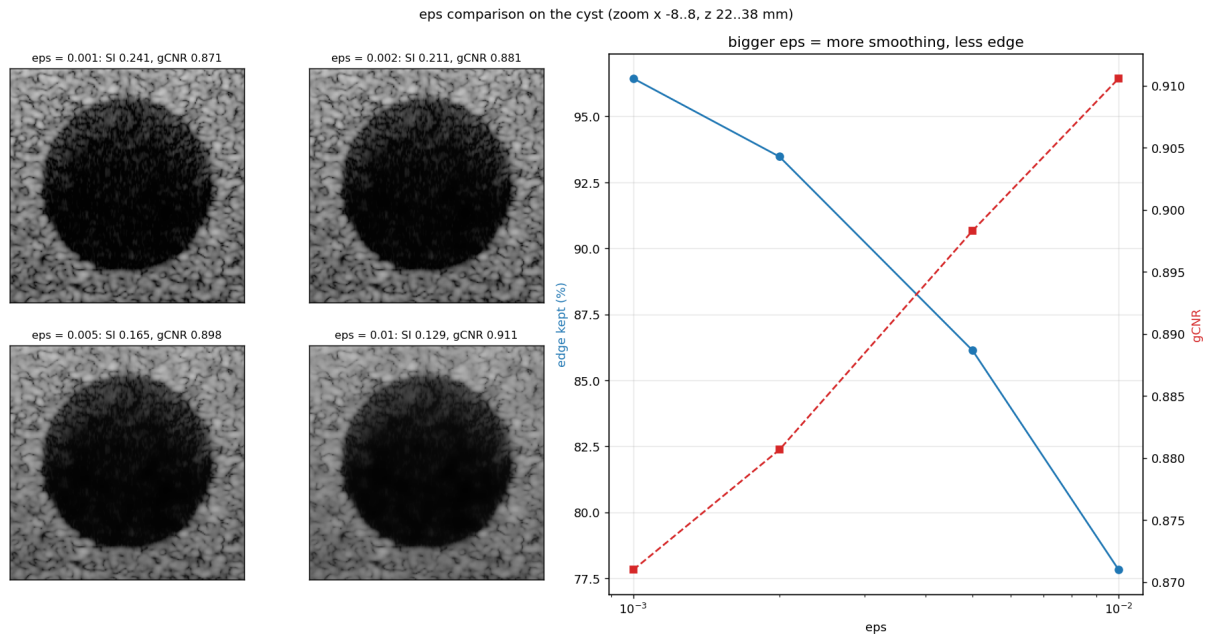
B3: after minus before. Changes are small (about ± 0.02 of full brightness, i.e. a few grey levels) and follow the speckle grains everywhere. Inside the cyst the darkest pixels are lifted slightly (red), which evens out the noise there.



B4: brightness along the row $z = 29.98$ mm through the cyst centre. The filtered line (red) follows the original but with smaller spikes; the steep step at the cyst edges (dashed lines at ± 6 mm) is kept.



B5: grey-level histograms. After filtering there are fewer extreme values: the spike at 0 (cyst) and the bright tail shrink, the histogram peak gets a bit higher.



B6: effect of eps. Larger eps smooths more (SI 0.241 -> 0.129) and raises gCNR (0.871 -> 0.911), but the edge sharpness kept falls from 96.4 % to 77.8 %. Only eps = 0.001 and 0.002 stay above the 90 % edge rule used in the study; the module uses 0.001.

eps	SI	gCNR	CNR	Edge kept
0.001	0.241	0.871	2.425	96.4 %
0.002	0.211	0.881	2.500	93.5 %
0.005	0.165	0.898	2.653	86.1 %
0.010	0.129	0.911	2.799	77.8 %

5.2 Console output

output/console_log.txt (real captured text)

```
=====
TEAM 8: POST-IMAGE PROCESSING (NITK-UsoundSim)
=====
Python 3.12.4, numpy 1.26.4, matplotlib 3.8.3
Folder: Team_8_Post_Image_Processing

=====
Post-Image Processing (code/post_processing.py)
=====
Note: project config.py not in this folder -> empty placeholder used
      (only scan_convert() needs it, and scan_convert() is not used in this demo).
Filter: postprocess(image) with its defaults -> method='guided', r=4, eps=0.001, gamma=1.0

cyst_lesion : input (512, 256), range 0.000-0.998 -> output range 0.004-0.984, time 5.3 ms
scatterers  : input (512, 256), range 0.000-0.998 -> output range 0.012-0.984, time 1.8 ms

Cyst lesion mask: circle r = 6 mm at (0, 30) mm -> 18418 px; background ring (10 px wide) -> 5162 px

Quality metrics (computed on uint8 images with the module's own functions):
image      metric                                before  after   change
-----
scatterers speckle index SI (lower = smoother)  0.178   0.163   -8.3%
cyst_lesion speckle index SI (lower = smoother)  0.308   0.241  -21.7%
cyst_lesion contrast lesion-ring (dB, 50 dB DR) -19.404 -19.317  +0.087
cyst_lesion CNR (higher = better)              2.323   2.425   +0.102
cyst_lesion gCNR (0..1, higher = better)        0.860   0.871   +0.010
cyst_lesion edge sharpness (Sobel, grey/px)    145.728 140.534  -5.194
cyst_lesion edge sharpness kept                  96.4%
```

Effect of eps (r = 4) on the cyst image:

eps	SI	gCNR	CNR	edge kept
0.001	0.241	0.871	2.425	96.4%
0.002	0.211	0.881	2.500	93.5%
0.005	0.165	0.898	2.653	86.1%
0.010	0.129	0.911	2.799	77.8%

Saving figures:

saved output/B1_before_after.png
 saved output/B2_cyst_edge_zoom.png
 saved output/B3_difference_map.png
 saved output/B4_brightness_profile.png
 saved output/B5_histograms.png
 saved output/B6_eps_comparison.png
 saved output/B_postprocessed_images.npz

=====

SUMMARY

=====

Cyst image: SI 0.308 -> 0.241, gCNR 0.860 -> 0.871, edge kept 96 %
 Files written to output/: 9 (including console_log.txt and console_screenshot.png)
 Total run time: 2.3 s

Terminal: python3 run_team8.py

=====

TEAM 8: POST-IMAGE PROCESSING (NITK-UsoundSim)

=====

Python 3.12.4, numpy 1.26.4, matplotlib 3.8.3
 Folder: Team_8_Post_Image_Processing

=====

Post-Image Processing (code/post_processing.py)

=====

Note: project config.py not in this folder -> empty placeholder used
 (only scan_convert() needs it, and scan_convert() is not used in this demo).
 Filter: postprocess(image) with its defaults -> method='guided', r=4, eps=0.001, gamma=1.0

cyst_lesion : input (512, 256), range 0.000-0.998 -> output range 0.004-0.984, time 5.3 ms
 scatterers : input (512, 256), range 0.000-0.998 -> output range 0.012-0.984, time 1.8 ms

Cyst lesion mask: circle r = 6 mm at (0, 30) mm -> 18418 px; background ring (10 px wide) -> 5162 px

Quality metrics (computed on uint8 images with the module's own functions):

image	metric	before	after	change
scatterers	speckle index SI (lower = smoother)	0.178	0.163	-8.3%
cyst_lesion	speckle index SI (lower = smoother)	0.308	0.241	-21.7%
cyst_lesion	contrast lesion-ring (dB, 50 dB DR)	-19.404	-19.317	+0.087
cyst_lesion	CNR (higher = better)	2.323	2.425	+0.102
cyst_lesion	gCNR (0..1, higher = better)	0.860	0.871	+0.010
cyst_lesion	edge sharpness (Sobel, grey/px)	145.728	140.534	-5.194
cyst_lesion	edge sharpness kept		96.4%	

Effect of eps (r = 4) on the cyst image:

eps	SI	gCNR	CNR	edge kept
0.001	0.241	0.871	2.425	96.4%
0.002	0.211	0.881	2.500	93.5%
0.005	0.165	0.898	2.653	86.1%
0.010	0.129	0.911	2.799	77.8%

Saving figures:

saved output/B1_before_after.png
 saved output/B2_cyst_edge_zoom.png
 saved output/B3_difference_map.png
 saved output/B4_brightness_profile.png
 saved output/B5_histograms.png
 saved output/B6_eps_comparison.png
 saved output/B_postprocessed_images.npz

=====

SUMMARY

=====

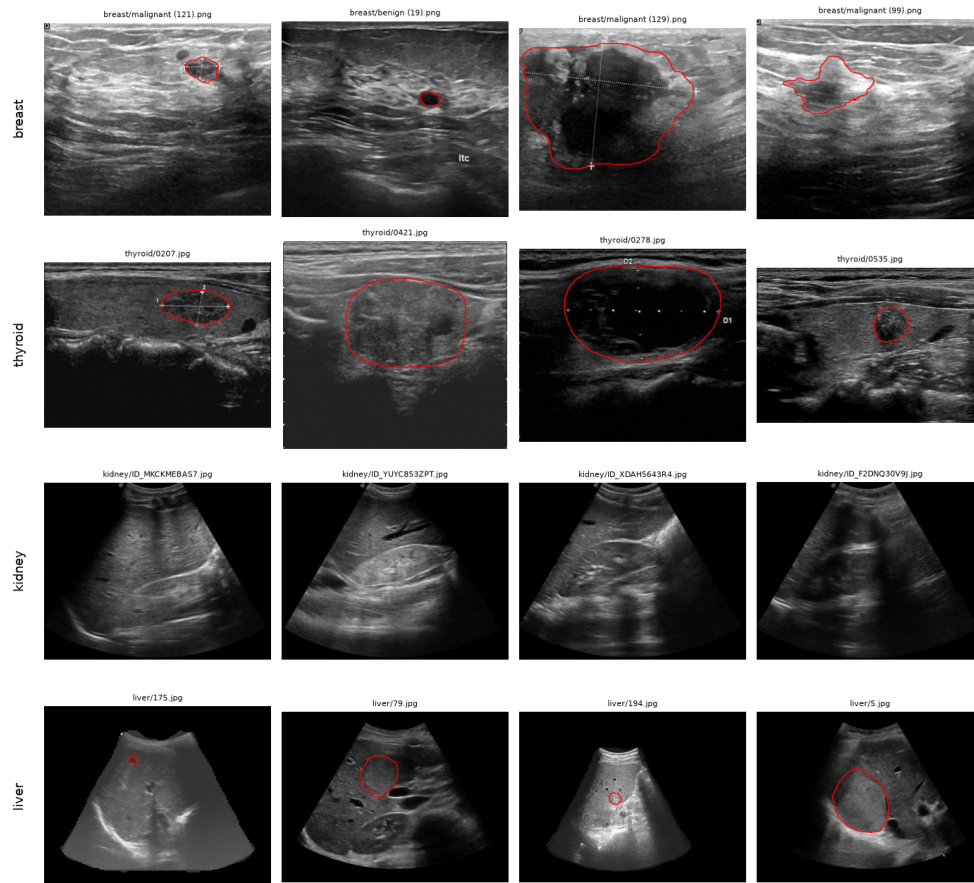
Cyst image: SI 0.308 -> 0.241, gCNR 0.860 -> 0.871, edge kept 96 %
 Files written to output/: 9 (including console_log.txt and console_screenshot.png)
 Total run time: 2.3 s

output/console_screenshot.png: the same console output as a picture.

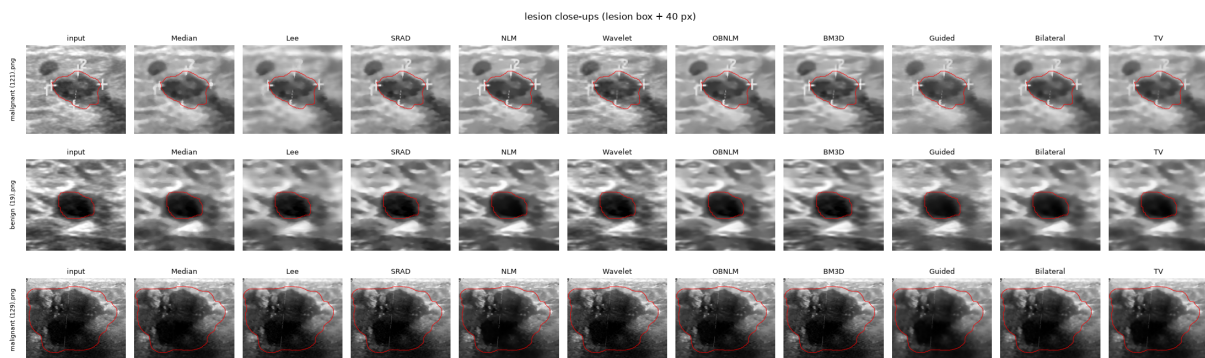
5.3 Results from the full study notebook (earlier run on real ultrasound datasets)

Important

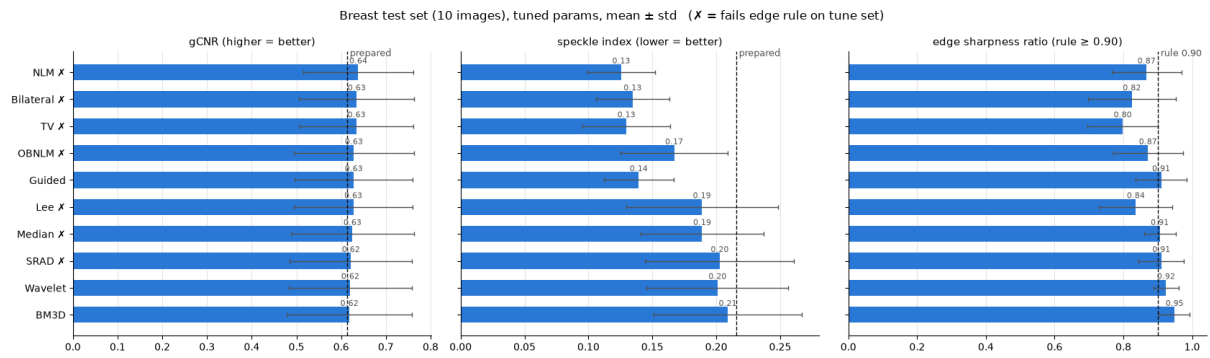
The figures in this section were NOT produced by run_team8.py. They were saved by the team's study notebook 262SP009_Post_Image_Processing_v2.ipynb in an earlier run on real clinical datasets and are stored in code/notebook_results/.



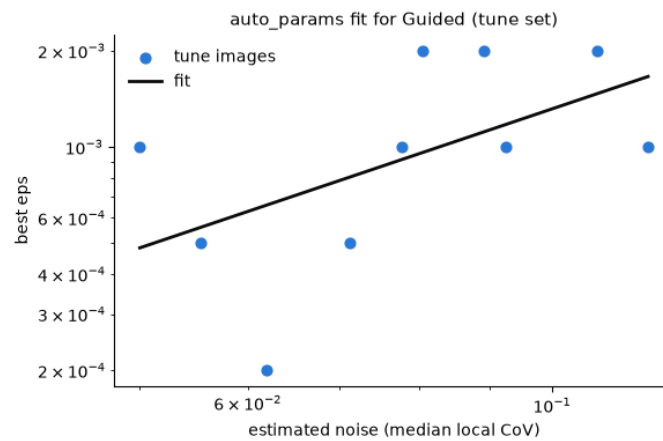
01_dataset_samples.png: sample images of the four organs used (breast BUSI, thyroid TN3K, kidney, liver); red = lesion masks.



04_filters_lesion_closeups.png: lesion close-ups of three breast images with all 10 filters at default settings.

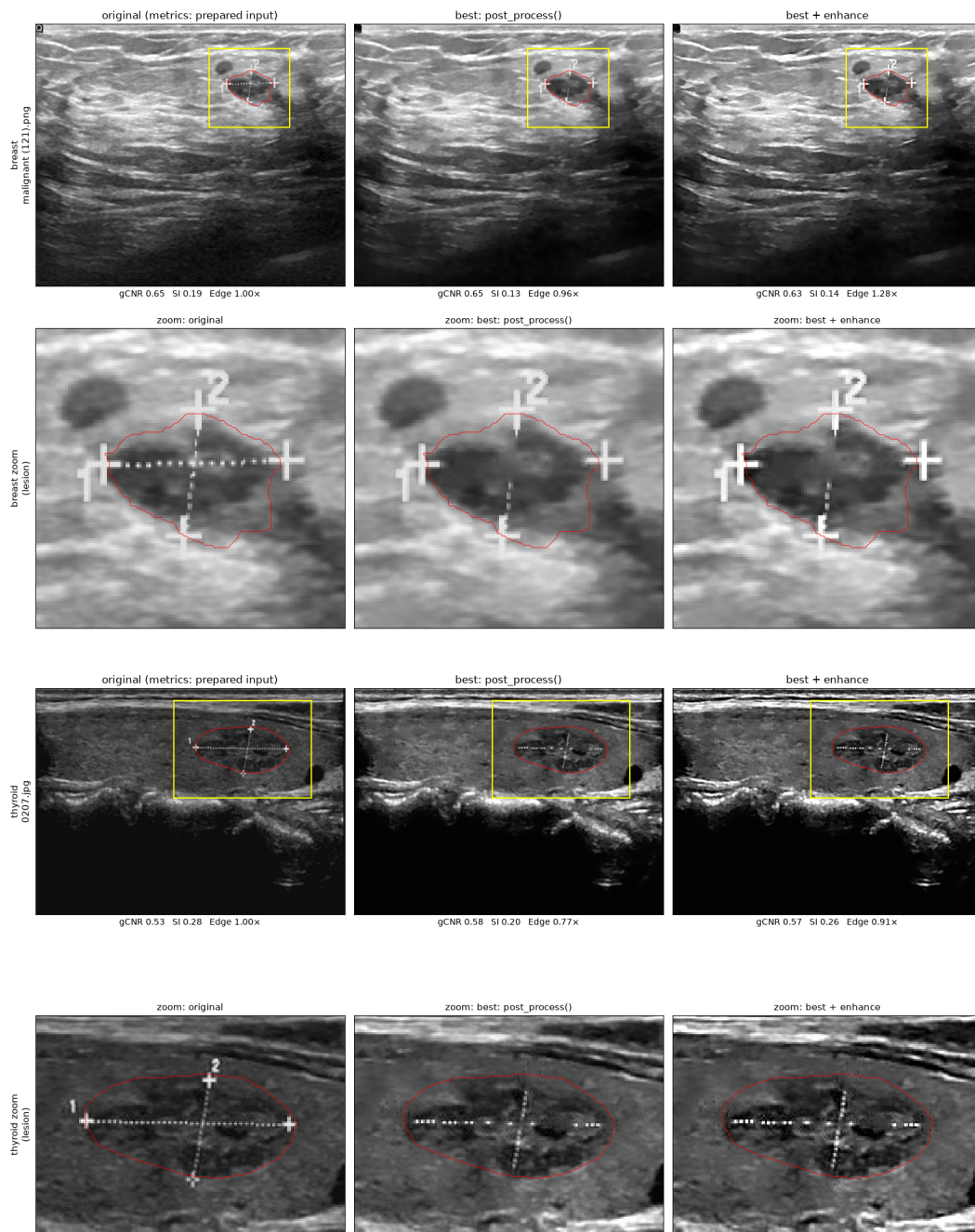


05_breast_bar_charts.png: breast test set (10 images), tuned parameters. gCNR, speckle index and edge ratio per filter; a cross marks filters that failed the 0.90 edge rule. The guided filter gives the lowest SI among the filters that pass the rule.



06_auto_params_fit.png: the attempt to choose eps automatically from the estimated noise level (weak fit, $r = 0.54$).

Final pipeline, one image per organ: full image + zoom of the yellow box (red = lesion / nodule; kidney has no mask)



07_final_all_organ_zoom.png (top part only, cropped for this page: breast and thyroid): original, post_process() and post_process + enhance, with zooms on the lesion.

How to run the study notebook in Google Colab

1. Go to <https://colab.research.google.com> and choose File > Upload notebook; pick code/262SP009_Post_Image_Processing_v2.ipynb.
2. Choose Runtime > Run all. The first cell installs kagglehub and bm3d; the data cells download the public datasets (internet needed, several hundred MB, some minutes).
3. Kidney images with masks need a free registration (explained in the notebook); without them it uses a fallback kidney set without masks automatically.
4. BM3D is slow (3-4 s per image), so the full notebook takes a while. Results appear under each cell.

6 How to run

Step by step, on macOS, Linux or Windows

1. Install Python 3.9 or newer from <https://www.python.org> (Windows: tick 'Add Python to PATH').
2. Open a terminal in the folder Team_8_Post_Image_Processing (macOS: right-click the folder in Finder > New Terminal at Folder; Windows: type cmd in the Explorer address bar).
3. Install the packages once: `python3 -m pip install -r requirements.txt` (Windows: `python -m pip install -r requirements.txt`).
4. Run: `python3 run_team8.py` (Windows: `python run_team8.py`). Do not run `code/post_processing.py` directly: it is a library module with no main block; `run_team8.py` imports it.
5. Look in output/: B1-B6 (figures), B_postprocessed_images.npz, console_log.txt and console_screenshot.png.

Expected runtime: about 3-8 seconds in total (this run: 2.3 s reported by the script). No internet is needed.

Opening the study notebook

- Colab: upload code/262SP009_Post_Image_Processing_v2.ipynb at <https://colab.research.google.com>. It needs internet and installs its own packages; see Section 5.3.

Troubleshooting

Problem	What to do
python3 / python not found	Install Python; on Windows try <code>py run_team8.py</code> .
No module named 'cv2'	<code>python3 -m pip install opencv-python</code>
No module named numpy / scipy / matplotlib	Repeat step 3 (<code>pip install -r requirements.txt</code>).
FileNotFoundError for input/ or code/	Keep the folder structure unchanged; run from inside the folder.
Message 'project config.py not in this folder'	Normal: only <code>scan_convert()</code> needs it and it is not used here.
pip says 'externally managed environment'	<code>python3 -m venv venv</code> ; <code>source venv/bin/activate</code> (Windows: <code>venv\Scripts\activate</code>); then step 3.
No plot windows appear	Normal: plots are saved as PNG files in output/.

7 Check yourself

Viva-style questions with short answers

Why does post-image processing come after B-mode image formation?

It works on the pixels of the finished grayscale image; the envelope, log compression and gray mapping must already be done.

What is speckle, and is it noise from the electronics?

A grainy interference pattern from many tiny scatterers adding with random phases; it is not electronic noise.

Why use a guided filter and not a Gaussian blur?

It smooths flat speckle regions but keeps edges, because $a \sim 1$ where the local variance is large; a blur softens edges too.

In the guided filter, what happens in a flat region and at an edge?

Flat: variance $\ll \epsilon$, $a \sim 0$, output $\sim$ local mean (smoothed). Edge: variance $\gg \epsilon$, $a \sim 1$, output $\sim$ input (edge kept).

What does $gCNR = 0.871$ mean, and what happens if ϵ is raised to 0.01?

The lesion and ring grey-level histograms overlap by only about 13 %. With $\epsilon = 0.01$, $gCNR$ rose to 0.911 and SI fell to 0.129, but only 77.8 % of the edge sharpness was kept, so the border got blurred.